QoS-Aware and Low-Cost Routing Optimization With Graph Reinforcement Learning in Hybrid Knowledge-Defined Networking

Yuqian Song, Jingli Zhou, Shudan Yu, and Jun Liu, *Member, IEEE*

***Abstract*— Internet service providers (ISPs) require modern networks to deliver high service satisfaction across diverse traffic types while maintaining minimal operational overhead. Traditional routing algorithms, although computationally efficient, rely on fixed heuristics and often fail to accommodate increasingly dynamic and heterogeneous network demands. In contrast, deep reinforcement learning (DRL)-based routing algorithms offer greater flexibility but typically introduce considerable computational cost. This issue is especially pronounced in prevalent centralized software-defined networking (SDN) architectures, resulting in excessive overhead and increased latency as the network grows. In this paper, we model a hybrid-traffic network scenario and propose a quality of service (QoS)-aware and low-cost routing algorithm based on graph reinforcement learning within a hybrid knowledge-defined networking (KDN) architecture. Routing decisions are jointly made by a global controller and distributed forwarding devices, with the unified objective of maximizing the minimum service satisfaction rate across the network. To reduce computational overhead and minimize disruptions from frequent route updates, we investigate two efficient implementation strategies: partial deployment (horizontal) and intermittent activation (vertical). Experimental results demonstrate that the proposed algorithm significantly outperforms benchmarks regarding the minimum service satisfaction rate, even under previously unseen topologies. Remarkably, our evaluation shows that 50% of the maximum achievable performance gain can be obtained through a lightweight implementation where just 28.5% of forwarding devices conduct policy inference, operating for only 60% of the total run time.**

***Index Terms*— Deep reinforcement learning, graph neural network, traffic engineering, routing optimization.**

I. Introduction

WITH the explosive growth of Internet traffic and the increasingly stringent quality-of-service (QoS) requirements of diversified applications, traffic engineering (TE) has attracted significant research attention over the past decade. TE generally involves three key tasks, including network information collection, routing optimization, and routing configuration, among which routing optimization serves as the core, determining efficient traffic distribution from sources to destinations. However, traditional interior gateway protocols (IGPs), such as RIP [1], OSPF [2], IS-IS [3], and Ecmp [4], are heavily dependent on static, pre-configured rules, which severely limit their adaptability to dynamic traffic patterns and evolving QoS requirements [5].

As an alternative, deep reinforcement learning (DRL) [6], an essential paradigm within machine learning, has demonstrated strong potential in addressing sequential decision-making problems. Unlike supervised and unsupervised learning, DRL eliminates the dependency on large-scale labeled datasets while maintaining a well-defined optimization objective, making it particularly suitable for network routing optimization. As a bio-inspired methodology, the DRL agent learns through iterative interactions with the environment in a trial-and-error manner. By accumulating experience over time, the agent can gradually refine its policy and ultimately enable adaptive and real-time derivation of optimal actions in response to dynamic network conditions.

With the advent of software-defined networking (SDN), which enables the practical deployment of complex routing policies in operational networks [7], DRL-based routing algorithms now have been systematically investigated by researchers, as first demonstrated by the work in [8]. The standard DRL-driven SDN architecture comprises three layers: *(i)* the data plane, consisting of forwarding devices; *(ii)* the control plane, which handles global network monitoring and DRL-based path computation; and *(iii)* the application plane, serving as an interface for high-level network applications. By decoupling control logic from forwarding elements, SDN facilitates the implementation of customized and sophisticated controller designs [9].

Given the rapid advancements of artificial intelligence (AI) in communications and networking, knowledge-defined networking (KDN) has been proposed as an evolution of SDN, integrating an AI-powered knowledge plane and an auxiliary management plane into the traditional SDN framework [10]. As research on routing optimization within the KDN paradigm remains in its infancy, and the core challenge in both KDN and SDN lies in the design of effective DRL-based routing agents, this work considers previous DRL-based routing algorithms, whether developed for KDN or SDN, as a unified category in subsequent analysis.

Although leveraging DRL for routing optimization in SDN and KDN has become an increasingly popular research direction, existing solutions still face several critical challenges. Some of these limitations are examined in our

Received 25 May 2025; revised 23 September 2025 and 6 November 2025; accepted 7 December 2025. Date of publication 10 December 2025; date of current version 13 January 2026. This work was supported in part by the National Natural Science Foundation of China under Grant 62371057. The associate editor coordinating the review of this article and approving it for publication was L. Zhou. *(Corresponding author: Jun Liu.)*

Yuqian Song, Jingli Zhou, and Jun Liu are with the Intelligent Perception and Computing Research Center, School of Artificial Intelligence, Beijing University of Posts and Telecommunications (BUPT), Beijing 100876, China (e-mail: songyuqian@bupt.edu.cn; wayzer@bupt.edu.cn; liujun@bupt.edu.cn).

Shudan Yu is with China Mobile Communications Corporation (e-mail: yushudan@chinamobile.com).

Digital Object Identifier 10.1109/TCCN.2025.3642320

2332-7731 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on January 18,2026 at 16:33:05 UTC from IEEE Xplore. Restrictions apply.

previous work [11], which highlighted the lack of consideration for network topological structures and poor generalization capability. To address these issues, [11] proposes a generalized routing algorithm employing graph reinforcement learning-based element-wise inference. In this work, we extend this investigation by identifying additional challenges from a more practical and deployment-oriented perspective.

First, the modern Internet operates as a hybrid traffic system, supporting multi-modal services across diverse vertical domains, such as web browsing, file transfer, high-definition video streaming, online gaming, and industrial control. These service flows coexist over shared network infrastructure and exhibit heterogeneous QoS requirements with varying levels of stringency. This inter-flow dynamics necessitate specialized modeling and processing strategies to realize cooperation and competition. Although differentiated services and QoS-aware routing have been longstanding research topics in traditional networking literature [12], [13], [14], [15], [16], [17], current DRL-based routing solutions largely adopt an oversimplified homogeneous flow treatment paradigm, as evidenced by works in [8], [18], [19], [20], [21], and [22]. Some pioneering studies have recognized the heterogeneity of network traffic [23], [24], however, these works either rely on additional networking technologies or fail to account for the coexistence of diverse flows in the same time-slot.

Second, most existing DRL-based routing solutions in SDN and KDN adopt a single-agent decision-making paradigm [8], [18], [19], [20], [21], [23]. While effective in small-scale networks, this centralized approach faces significant scalability challenges as network size scales up, incurring high computational overhead and prolonged reaction times. To address scalability limitations, recent studies [22], [24] have explored the multi-agent deep reinforcement learning (MADRL) paradigm. However, [24] co-locates all agents in the control plane, retaining the inherent drawbacks of centralized control (e.g., single-point crisis); [22] distributes intelligence across network devices but lacks global optimization capabilities, potentially compromising performance for flows with stringent QoS requirements and reducing fault awareness.

Third, TE inherently conducts a periodic redistribution of network traffic. However, frequent flow rerouting operations may cause significant QoS degradation and service disruptions [25], and the corresponding updates of forwarding entries impose substantial management overhead and introduce prolonged delays [26]. To this end, prior studies [25], [26], [27] propose leveraging DRL to identify critical flows and formulate their routing as a linear programming (LP) optimization problem to determine optimal split ratios. While experimental results demonstrate that rerouting only a subset of flows can achieve near-optimal performance, their reliance on LP solvers prevents the network from achieving full autonomy and results in low computational efficiency in large-scale scenarios.

In this work, we aim to propose a QoS-aware and low-cost routing algorithm based on graph reinforcement learning in a hybrid KDN architecture. To address the aforementioned first challenge, we model the network as a hybrid traffic system and develop an adaptive sacrifice mechanism to optimize the minimum satisfaction rate across diverse service types. For the second challenge, a hybrid KDN architecture is constructed, where the routing policy is jointly determined by a global-view centralized controller and local-perspective forwarding devices. To overcome the third challenge, we investigate two efficient implementation strategies, partial deployment (horizontal) and intermittent activation (vertical), to minimize unnecessary flow rerouting operations while maintaining network performance. In contrast to the aforementioned studies [25], [26], [27], our work focuses on intermittently activating a subset of forwarding devices to apply DRL-based strategies, rather than identifying critical flows. The main contributions of this paper are as follows:

TABLE I
COMPARISON OF RELATED WORKS

Pub.	Traffic	Agent	Feature	Partial Routing
[8]	Single	Single-agent	✗	✗
[18]	Single	Single-agent	✗	✗
[19]	Single	Single-agent	CNN-based	✗
[20]	Single	Single-agent	RNN-based	✗
[21]	Single	Single-agent	✗	✗
[22]	Single	Multi-agent	✗	✗
[23]	Mixed	Single-agent	GNN-based	✗
[24]	Mixed	Multi-agent	✗	✗
[25]	Single	Single-agent	GNN-based	✓
[26]	Single	Single-agent	✗	✓
[27]	Single	Single-agent	CNN-based	✓
[11] (Prior)	Single	Single-agent	GNN-based	✗
Ours	**Mixed**	**Single-multi-agent**	**GNN-based**	✓

- We formulate a QoS-aware and low-cost intra-domain probabilistic routing problem tailored for hybrid-traffic autonomous systems. Unlike conventional objectives that emphasize individual service satisfaction, our formulation focuses on long-term maximization of the minimum satisfaction rate across all service types, ensuring fairness in service delivery.
- We introduce an innovative hybrid KDN architecture that combines centralized and distributed intelligence with customized observation scopes, operating under a counterfactual-baseline comparative training framework. Each agent employs a graph transformer [36]-based feature extraction network and follows element-wise action inference principles. To enable adaptive inter-service cooperation and competition, we develop a sacrifice-supported action space and a Tchebycheff [29]-based reward function. Furthermore, we implement a two-phase extended training process to yield a supervised learning-based classifier for activation timing and a SAGPool [30]-based selector for critical nodes, collectively minimizing routing updates while maintaining performance.
- We conduct extensive evaluations against benchmarks on both training and unseen network topologies. Simulation results show that our algorithm consistently outperforms benchmarks in terms of minimum service satisfaction rate and demonstrates strong generalization capability.

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on January 18,2026 at 16:33:05 UTC from IEEE Xplore. Restrictions apply.

Notably, up to 50% of the maximum performance gain can be achieved through a lightweight implementation where only 28.5% of the forwarding devices conduct policy inference and routing update for just 60% of the run time, significantly reducing operational costs.

The remainder of this paper is organized as follows. Section II reviews the related works in SDN and KDN. Section III describes the routing problem formulated in this work. Section IV introduces the architecture and algorithm design. Section V details the training and deployment strategy of the proposed DRL-based routing solutions. Section VI presents the performance evaluation of the proposed algorithms. Finally, Section VII concludes this paper.

II. Related Works in SDN and KDN Architectures

A. Elementary DRL-Based Routing Algorithms

The work in [8] represents the first DRL-based routing algorithm designed for SDN, where the agent aims to minimize the mean network delay by dynamically adjusting link weights in response to the traffic matrix. Inspired by [8], the work in [18] proposes a more advanced DDPG-based routing algorithm featuring a configurable multi-objective utility function that co-optimizes delay, packet loss, and throughput. Subsequent works such as [19] and [20] further enhance routing intelligence by integrating convolutional neural networks (CNNs) and recurrent neural networks (RNNs), respectively, enabling more sophisticated network state characterization for improved decision-making. In contrast to these previous works that generate link weights from traffic matrices, the agent in [21] learns to select optimal paths from precomputed shortest-path candidates based on source-destination pairs. In addition, the work in [21] establishes a comprehensive KDN architecture comprising five functional modules: topology, monitoring, processing, routing, and installation, along with a detailed operation workflow. In alignment with the KDN architecture, the topology module represents the data plane, the monitoring and installation modules operate within the control plane, the processing module corresponds to the management plane, and the routing module embodies the knowledge plane.

B. Advanced DRL-Based Routing Algorithms

While prior works adopt a single-agent decision-making paradigm that suffers from scalability limitations [8], [18], [19], [20], [21], the work in [22] employs multiple device-side agents to infer traffic split ratios across candidate shortest paths. Recognizing that modern applications often demand diverse and stringent QoS requirements, and that conventional routing approaches without flow-specific QoS characteristics may degrade network performance, the work in [24] proposes an online MADRL-based routing scheme that accommodates multi-type service requirements. Each router is managed by an agent residing on the controller, tasked with next-hop selection for flow-specific QoS utility maximization. However, this work assumes that the flow arrival interval exceeds the route installation time, which may require additional filter operations in high concurrency scenarios. Alternatively, the work in [23] adopts the network slicing technology [31], which allows the substrate bandwidth to be partitioned into isolated logical network slices, each dedicated to a single service type. In this case, the controller-side agent must simultaneously optimize routing paths for all network slices and their respective bandwidth allocations, introducing operational complexity that grows exponentially in intricate networks.

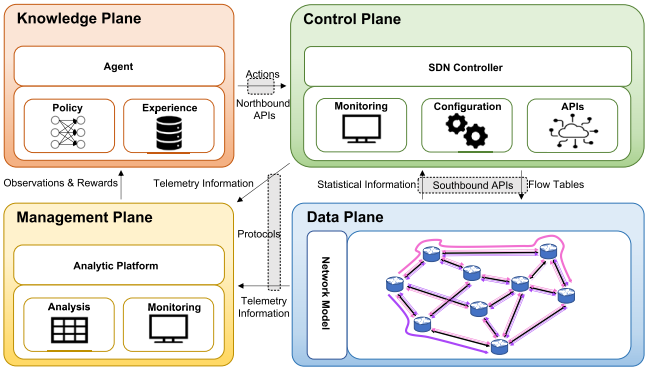


Fig. 1. Illustration of a standard DRL-based KDN architecture.

The works in [25], [26], and [27] primarily investigate critical flow selection. The foundational work [27] employs a traffic matrix as the state input to identify critical flows, and utilizes the reciprocal of maximum link utilization (MLU) as the reward signal. On this basis, the work in [25] enhances the state representation to include topological connectivity, historical traffic matrices, and past routing strategies, while formulating a composite reward function combining both MLU-based performance metrics and network disturbance indicator (quantified as the fraction of total traffic rerouted during updates). In contrast, the work in [26] adopts a novel perspective by focusing on critical forwarding entry selection. Beyond selection, [26] introduces an extended training phase to explicitly balance the quantity of critical entries against overall network performance. However, all three approaches ultimately depend on LP solvers for route computation, which may limit their scalability in large-scale dynamic networks.

III. System Model

A. KDN Architecture

Fig. 1 illustrates the standard KDN architecture. In the following, we will explain the functionalities of each building block.

1) Data Plane: The data plane is designed based on an autonomous system network model, comprising a set of programmable forwarding devices (e.g., Open vSwitch [32]) and interconnecting links.

2) Control Plane: The control plane acts as a pivotal intermediary in KDN. Through southbound APIs (e.g., OpenFlow [33]), it collects network statistics to maintain a global view and enforces flow rules on forwarding devices. Via northbound APIs (e.g., RESTful [34]), it interacts with the knowledge plane, obtains optimized routing policies, and translates them into flow table updates.

3) Management Plane: The management plane operates orthogonally to the data and control planes, using protocols

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on January 18,2026 at 16:33:05 UTC from IEEE Xplore. Restrictions apply.

TABLE II
NOTATIONS INTRODUCED IN SECTION III (LISTED IN ORDER OF APPEARANCE)

Notation	Definition
$\mathcal{G}$	The weighted directed network topology graph
$\mathcal{N}$	The n-th node and the set of forwarding nodes
$\mathcal{L}$	The set of directed links
$\mathcal{W}$	The set of associated link weights
N, L	The number of nodes and the number of links
k_0, k_1, k_2, k_3	The set of service types
p_i	The packet size of service type k_i
d_i	The delay constraint of service type k_i
p_{avg}	The average packet size across all service types
λ	The flow rate
I_{avg}	The average packet generation interval
$d(f,i)$	The delay of flow f of service type k_i
l_{mn}	The link from node $m \in \mathcal{N}$ to node $n \in \mathcal{N}$
$\mathcal{P}_f$	The complete routing path of flow f
$d_{p,mn}$	The propagation delay of link l_{mn}
$d_{q,m}$	The queuing delay at node $m \in \mathcal{N}$
c_{mn}	The bandwidth of link l_{mn}
τ	The statistical interval
$\mathcal{F}_{i,all}$	The set of all flows of service type k_i
σ_f	The delivery indicator of flow f
$A(f)$	The delivery set of flow f
$\mathcal{F}_{i,arr}$	The delivery set of service type k_i
$\mathcal{F}_{i,sat}$	The set of timely-delivered flows of service type k_i
η_i	The satisfaction rate of service type k_i
u_{mn}	The utilization of link l_{mn}

such as NetFlow [35] to collect telemetry for monitoring and analytics. It enhances long-term performance and reliability by collaborating with the control plane to provide the knowledge plane with a comprehensive network-wide view.

4) Knowledge Plane: Since DRL-based algorithms are computation-intensive, the agent is decoupled from the control plane and deployed in the knowledge plane to avoid performance degradation. Leveraging network information collected by the management plane, it performs reasoning and decision-making, reconfigures forwarding rules via the controller, and thus optimizes routing. As the embodiment of "knowledge" in KDN, the agent is central to system performance. While the conventional knowledge plane interacts only with the control plane, we propose a hybrid KDN architecture in which it also collaborates with the data plane for decision-making. Details of the agent design are presented in Section IV.

B. Network Model

The data plane in Fig. 1 illustrates the probabilistic routing scenario considered in this work. The network is modeled as a weighted directed graph $\mathcal{G} = \{\mathcal{N}, \mathcal{L}, \mathcal{W}\}$, where $\mathcal{N}$ represents the set of forwarding nodes, $\mathcal{L}$ denotes the set of directed links, and $\mathcal{W}$ corresponds to the set of associated link weights. The network scale is characterized by $N = |\mathcal{N}|$ nodes and $L = |\mathcal{L}|$ links. Each node maintains a dedicated packet buffer with uniform capacity, operating under a first-in-first-out (FIFO) queuing discipline. All links exhibit identical propagation delays and uniform bandwidths. During transmission, flows that exceed the instantaneous forwarding capacity are temporarily buffered. To mitigate excessive queuing and avoid potential buffer overflows, which can severely impair the forwarding efficiency, we assume that any packet residing in the system for more than $2s$ or surpassing its predefined time-to-live (TTL) is discarded. In the context of probabilistic routing, nodes determine next-hop selections for incoming flow requests based on probability distributions derived from link weights.

Without loss of generality, we assume the network is a multi-source and multi-destination transmission scenario, where each node may serve as a source or a destination. To model traffic heterogeneity, we define 4 distinct service types, denoted as k_0, k_1, k_2, k_3, each characterized by a unique packet size p_i and a specific delay constraint d_i. To be specific, we categorize these service types into two behavioral classes based on flow generation patterns: *(i)* persistent service flows (k_0, k_1), which maintain continuous transmission with durations uniformly distributed in $[10\ s, 20\ s]$; *(ii)* intermittent service flows (k_2, k_3), which exhibit bursty transmission patterns, alternating between short active periods (uniformly distributed in $[2\ s, 4\ s]$) and dormant intervals (also uniformly distributed in $[2\ s, 4\ s]$). Furthermore, we assume that k_0 and k_2 have moderate QoS requirements, whereas k_1 and k_3 impose stricter QoS constraints, introducing differentiation in QoS levels among flows within the same behavioral class. In comparison, k_3 has the most strict QoS requirements.

To reflect dynamic traffic behavior in real-world networks, each source node randomly switches both the flow's destination and service type for every newly generated flow. All destinations and service types are selected with equal probability. Let p_{avg} denote the average packet size across all service types. The average packet generation interval is $I_{avg} = \frac{p_{avg}}{\lambda}$, where λ represents the flow rate. The actual packet generation interval for each source node is drawn from a uniform distribution.

During transmission, the end-to-end delay comprises propagation delay, queuing delay, and transmission delay, while the processing delay is considered negligible in this work. Accordingly, the total delay experienced by a flow f of service type k_i is expressed as

$$d(f,i) = \sum_{l_{mn} \in \mathcal{P}_f} \left[d_{p,mn} + d_{q,m} + \frac{p_i}{c_{mn}} \right], \tag{1}$$

where l_{mn} denotes a link traversed by flow f, $\mathcal{P}_f$ represents the complete routing path of f, $d_{p,mn}$ is the propagation delay of link l_{mn}, $d_{q,m}$ is the queuing delay at node m, and c_{mn} is the bandwidth of link l_{mn} used to compute the transmission delay.

To facilitate evaluation, we define a statistical interval of τ. Let $\mathcal{F}_{i,all}$ denote the set of all flows of service type k_i generated within the current statistical interval. Each flow $f \in \mathcal{F}_{i,all}$ is associated with a binary indicator σ_f, defined as

$$\sigma_f = \begin{cases} 1, & \text{if } \ A(f) \neq \emptyset, \\ 0, & \text{otherwise}, \end{cases} \tag{2}$$

where $A(f)$ denotes the set of packets of flow f that have successfully arrived at the destination. Based on this, the set of successfully arrived flows of type k_i is represented as $\mathcal{F}_{i,arr} = \{\forall f \in \mathcal{F}_{i,all} | \sigma_f = 1\}$. To evaluate service satisfaction, we further define the set $\mathcal{F}_{i,sat} \subseteq \mathcal{F}_{i,arr}$ to include those

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on January 18,2026 at 16:33:05 UTC from IEEE Xplore. Restrictions apply.

flows whose total delay satisfies $d(f,i) \leq d_i$. The satisfaction rate for service type k_i is calculated as

$$\eta_i = \frac{|\mathcal{F}_{i,sat}|}{|\mathcal{F}_{i,all}|}, \tag{3}$$

C. Problem Formulation

Given the co-existence of heterogeneous service flows, resource competition is inevitable, leading to trade-offs among the satisfaction rates of different service types. A common approach to address this issue is to aggregate these satisfaction rates into a single optimization objective via weighted summation. However, selecting appropriate weights is challenging, as they can be highly subjective and prone to bias. Moreover, tuning these weight parameters typically requires extensive simulation, which is both time-consuming and computationally intensive. Consequently, this constitutes a complex multi-objective optimization problem.

Among the various methods proposed to decompose a multi-objective optimization problem into a set of scalar optimization sub-problems, the Tchebycheff decomposition approach is the most widely adopted. This is due to its ability to effectively handle multi-objective problems with non-convex Pareto fronts. In addition, the reasoning behind this approach is intuitive and easily interpretable, making it more amenable to integration with other algorithms. The formal formulation of the Tchebycheff function is given by

$$\max_{\mathbf{x}\in\Omega}(f_0(\mathbf{x}),\ldots,f_3(\mathbf{x}))^{\mathrm{T}} = \min_{0\leq i\leq 3}\left\{|\mathbf{z}_i^* - f_i(\mathbf{x})|\right\}, \tag{4}$$

where $\mathbf{x}$ represents the vector of decision variables, Ω represents the feasible solution space, $f_i(\mathbf{x})$ represents the satisfaction rate for service type k_i, and $\mathbf{z}_i^*$ is the ideal satisfaction rate for service type k_i.

Following the aforementioned transformation, the overall optimization objective is effectively refined from a broad goal of maximizing network performance to a more concrete aim of optimizing the network's baseline performance. Specifically, it focuses on minimizing the gap between the satisfaction rates of various service types and the ideal 100% satisfaction rate. On this basis, the optimization objective of this work is mathematically formulated as

$$\min_{\mathcal{W}^t} \lim_{T\to\infty} \frac{1}{T}\sum_{t=1}^{T} \max_{0\leq i\leq 3}\left\{1-\eta_i^t\right\}, \tag{5a}$$

$$\text{s.t. } 0 \leq w^t \leq 1,\ \forall w^t \in \mathcal{W}^t,\ \forall t \in T, \tag{5b}$$

$$d(f,i)^t \leq d_i, \quad 0 \leq i \leq 3, \tag{5c}$$

$$0 \leq u_{mn}^t \leq 1, \quad \forall l_{mn} \in \mathcal{L}, \tag{5d}$$

where $\mathcal{W}^t$ denotes the joint link weights at time step t, T represents the duration of the routing task.[1] (5b) restricts the original link weights to the interval $[0,1]$. Notably, a subsequent transformation detailed in Section IV-C permits the adjusted link weights to exceed 1 for downstream decision processes. (5c) guarantees that each flow meets its specified delay requirement; otherwise, it is deemed unsuccessful. (5d) enforces the link capacity limitation, where u_{mn}^t denotes the utilization of link l_{mn} and must not surpass 1.

[1]Here, $T \to \infty$ follows the general infinite-horizon formulation. The same objective applies under a finite-horizon setting with fixed T.

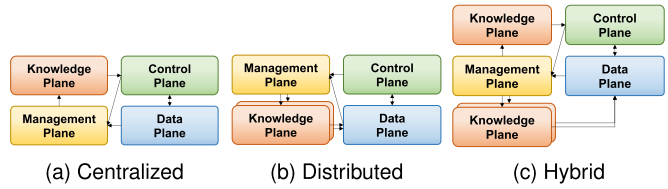


Fig. 2. Illustrations of different KDN architecture.

To summarize, this work aims to minimize the maximum deviation of the satisfaction rates from their ideal value of 1 across all service types, i.e., maximizing the minimum satisfaction rates across all service types. Given the dynamic nature of network environments and the large-scale variables, the optimization problem formulated in (5) is inherently complex and computationally challenging. To this end, this work introduces a DRL-based algorithm to enable the development of adaptive and QoS-aware routing policies, which offers the capability to learn effective decision-making in dynamic environments through continuous interaction and feedback. In the subsequent sections, we present a comprehensive description of the proposed DRL-based algorithm.

IV. Proposed Algorithm: Architecture and Algorithm Design

This section first introduces the key components of the proposed DRL-based algorithm, encompassing the agent, observation space, action space, and reward function. Then, we detail the graph transformer-based feature extraction module.

A. Agent

In this work, the definition of agents involves two critical aspects. The first concerns the deployment of knowledge, i.e., where the agents reside within the network. The second relates to whether the agent operates in a node-centric or link-centric manner under the graph transformer framework.

Regarding the first aspect, we propose a hybrid KDN architecture that empowers both the centralized controller in the control plane and the distributed forwarding devices in the data plane with intelligent routing capabilities. This design aims to maintain logically centralized control while alleviating the scalability limitations caused by the computational burden of a single control point, serving as a compromise between fully centralized KDN and fully distributed KDN architectures (where each device is linked to a dedicated knowledge plane). Fig. 2 illustrates these three distinct KDN architectures.

Besides, our system model incorporates two behavioral patterns of service flows: persistent service flows (k_0, k_1) and intermittent service flows (k_2, k_3). Within each category, the two flow types are further distinguished by their respective QoS stringency, and k_3 is associated with the most strict QoS requirement. Correspondingly, we assume that local forwarding devices handle frequent events associated with k_0 and k_1, while the centralized "root" controller is responsible for

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on January 18,2026 at 16:33:05 UTC from IEEE Xplore. Restrictions apply.

infrequent events k_2 and k_3, especially those that may require a global view for higher QoS.

As for the second aspect, our previous work [11] has demonstrated the superior performance of the link-centric paradigm over the node-centric approach through comprehensive experimental validation. Therefore, this work adopts a link-centric agent modeling, where each link is managed by an agent deployed at its originating node. The graph transformer-based inter-agent message passing proceeds following the digraph as established in [36]. In detail, each vertex in the digraph represents a link in the physical topology, and two directed links, $u \to v$ and $w \to x$, are connected if $v = w$.

B. Observation

Considering the multi-source and multi-destination transmission environment, we define the original feature of each link $l_{mn} \in \mathcal{L}$ regarding destination D at time step t as

$$h^t_{mn,D} = \left\{\Delta_{mn,D}, u^t_{mn}\right\}, \tag{6}$$

where $\Delta_{mn,D}$ is a one-hot vector of length 3 that encodes the hop difference between node m and node n relative to the current destination node D, and u^t_{mn} denotes the normalized utilization of link l_{mn} that reflects both network-side traffic demands and agent-side policy effectiveness. Since any node in the network may serve as a destination, each link l_{mn} is associated with a feature set of dimension $[N,\ 4]$, where N is the total number of nodes in the network.

Prior to action inference, agents perform feature extraction through the graph transformer network. The controller-side (centralized) agent has access to the raw features of all links across the network and performs global message passing over the pre-constructed digraph. The observation of this agent at time step t is defined as

$$o^t_c = \left\{H^t_{mn,D}\right\}, \forall l_{mn} \in \mathcal{L}, \forall D \in \mathcal{N}, \tag{7}$$

where $H^t_{mn,D}$ represents the high-level abstracted feature of link l_{mn} after global-level graph proccessing. In contrast, for the forwarding device-side (distributed) agents, we consider practical constraints in real-world communication environments and assume that each agent can only access the features of its one-hop neighboring agents. Under this setting, the observation of a distributed agent associated with link l_{mn} is given by

$$o^t_{mn} = \left\{\hat{H}^t_{mn,D}\right\}, \forall D \in \mathcal{N}, \tag{8}$$

where $\hat{H}^t_{mn,D}$ represents the locally abstracted feature.

C. Action

Corresponding to the solution of the optimization problem (5) in Section III-C, the inferred action at time step t is

$$A^t = \left\{w^t_{i,mn,D}\right\}, 0 \le i \le 3, \forall l_{mn} \in \mathcal{L}, \forall D \in \mathcal{N}, \tag{9}$$

where $w^t_{i,mn,D} \in [0,1]$ denotes the link weight assigned to l_{mn} for service type k_i with respect to destination node D at time step t. The dimensionality of the action space is $[N, L, 4]$. This action set is directly produced by the DRL agent.

Within the hybrid decision-making framework, the action of the centralized controller agent at time step t is defined as

$$a^t_c = \left\{w^t_{i,mn,D}\right\}, 2 \le i \le 3, \forall l_{mn} \in \mathcal{L}, \forall D \in \mathcal{N}, \tag{10}$$

while the action of a distributed agent associated with link l_{mn} is given by

$$a^t_{mn} = w^t_{i,mn,D}, 0 \le i \le 1, \forall D \in \mathcal{N}. \tag{11}$$

The complete action set is therefore expressed as

$$A^t = a^t_c || _{\forall l_{mn} \in \mathcal{L}}\ a^t_{mn}, \tag{12}$$

where $||$ denotes concatenation.

In contrast to conventional link weight-based routing approaches that demand auxiliary algorithms, e.g., Dijkstra's algorithm, to compute forwarding paths, our probabilistic routing paradigm interprets link weights as the probability of a flow traversing the corresponding link, which maintains a scalable hop-by-hop routing manner. Neighbors connected via links with the highest probabilities are selected as the next hops. Since each forwarding node typically has multiple outgoing links, the raw weights associated with all adjacent links must be normalized before making the final forwarding decision. For adaptive cooperation and competition among different traffic types, we further extend the action space to incorporate packet sacrifice. The specific modifications are as follows.

First, one potential limitation affecting the effectiveness of probabilistic routing arises in soft values. For example, in lightly loaded networks where extreme "0-1" link weights are sufficient to achieve near-optimal performance, the continuous and non-binary outputs of deep neural networks may introduce redundant probabilities, resulting in imprecise forwarding decisions. To address this issue, we incorporate an exponential mapping function to sharpen the distinctions among link weights, enabling agents to better interpret the output values, where those close to 0 indicate a strong tendency to avoid the corresponding link, while values approaching 1 imply a strong forwarding preference. In this step, the mapped weight is given by

$$\hat{w}^t_{i,mn,D} = e^{\beta \cdot w^t_{i,mn,D}}, \tag{13}$$

where β is a tunable parameter controlling the dispersion of the resulting weight distribution. To avoid the mapped weights clustering near zero or reaching excessively large values, a moderate choice of β is required to ensure a balanced steepness.

Second, to facilitate coordination among heterogeneous service flows and maximize the minimum satisfaction rate across all service types, we propose a packet sacrifice mechanism. This mechanism is motivated by the observation that flows with stringent QoS requirements often suffer lower satisfaction rates, while those tolerant to performance variations typically achieve higher satisfaction. By strategically discarding a portion of less-sensitive traffic, the network can reserve more resources for performance-critical services. To delineate the

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on January 18,2026 at 16:33:05 UTC from IEEE Xplore. Restrictions apply.

boundary between normal forwarding and packet sacrifice, we then refine the weight mapping as follows:

$$\hat{w}^t_{i,mn,D} = e^{\beta\cdot(w^t_{i,mn,D}-\zeta)}, \tag{14}$$

where $\zeta \in [0,1]$ denotes the sacrifice threshold. For weights $w^t_{i,mn,D} \leq \zeta$, the mapped values satisfy $\hat{w}^t_{i,mn,D} \leq 1$, indicating standard forwarding behavior without packet sacrifice. In contrast, when $w^t_{i,mn,D} > \zeta$, $\hat{w}^t_{i,mn,D}$ exceeds 1, and a fraction of packets is intentionally dropped, with the sacrifice ratio computed as $\frac{\hat{w}^t_{i,mn,D}-1}{\hat{w}^t_{i,mn,D}} \times 100\%$. To ensure that packet sacrifice is only triggered in selective cases, we set $\zeta = 0.9$. On this base, the maximum sacrifice ratio is $\frac{e^{0.1\beta}-1}{e^{0.1\beta}}$. To excessive packet loss that could destabilize network operations, we eventually select $\beta = 10$, which bounds the maximum sacrifice ratio to $\frac{e-1}{e} \approx 63.2\%$.

Note that while the original weights $w^t_{i,mn,D}$ are bounded within $[0,1]$, the mapped weight $\hat{w}^t_{i,mn,D}$ fall within the range of $[e^{-9}, e]$. Importantly, this extended range does not affect forwarding decisions, as the next-hop selection is based on the normalized relative values of all outgoing link weights. The absolute magnitude of $\hat{w}^t_{i,mn,D}$ serves exclusively to determine whether packet sacrifice is triggered and to quantify the corresponding sacrifice ratio.

Besides, while low-demand flows exhibit greater tolerance to performance degradation and can thus accommodate partial packet sacrifice, unconstrained discarding remains undesirable. To regulate this behavior, we propose the Tchebycheff-based reward function for "adaptive" sacrifice, which facilitates the competition among heterogeneous services.

D. Reward

In line with the optimization objective in (5), we define the fundamental reward at time step t as

$$r^t = -\max_{0\leq i\leq 3}\left\{1-\eta^t_i\right\}. \tag{15}$$

In this formulation, the reward is not tied to any specific service type, instead, it is determined by the least-satisfied flow at each time step that can be any service type. Therefore, this reward structure discourages both excessive and insufficient sacrifice, as either will negatively impact the overall reward. When combined with the compromises enabled by the action space, this reward function acts as a safeguard, enabling agents to dynamically regulate packet sacrifice behaviors in response to evolving network conditions.

E. Graph Transformer-Based Feature Extraction

To model the complex relationships among forwarding devices embedded in the non-Euclidean graph topology, we adopt a graph transformer network before formal action inference. Among various graph neural network (GNN) paradigms, the graph transformer is selected for its multi-head self-attention mechanism, derived from the standard transformer architecture, and the decoupled learning of query, key, and value components, which collectively facilitates highly flexible and adaptive information aggregation, making it one of the most expressive architectures among state-of-the-art GNNs.

The core of graph transformer-based feature extraction is layer-wise message passing. Each graph transformer (GT) layer performs 1-hop neighborhood information propagation and feature updating. Let $h^t_{mn,D}$ denote the initial feature of vertex l_{mn} in the link-centric digraph. The attention coefficient $\alpha_{mn,ij,D}$ between vertex l_{mn} and l_{ij} in $head_a$ is computed as

$$q_{mn,a} = W_{q,a}h^t_{mn,D},\ k_{ij,a} = W_{k,a}h^t_{ij,D}, \tag{16}$$

$$\alpha_{mn,ij,D,a} = softmax(\frac{q^T_{mn,a}k_{ij,a}}{\sqrt{d_a}}), \tag{17}$$

where $W_{q,a}$ and $W_{k,a}$ are trainable weight matrices. d_a represents the hidden size of $head_a$. The updated hidden feature of vertex l_{mn} in $head_a$ is then computed as

$$H^t_{mn,D,a,0} = W_{v_1,a}h^t_{mn,D} + \sum_{l(i,j)\in\Gamma(mn)} \alpha_{mn,ij,D,a}W_{v_2,a}h^t_{ij,D}, \tag{18}$$

where $W_{v_1,a}$ and $W_{v_2,a}$ are learnable transformation matrices. $\Gamma(mn)$ represents the 1-hop neighborhood of vertex l_{mn} in the digraph. Finally, the outputs from all attention heads are concatenated, yielding the updated hidden feature of vertex l_{mn} at the first GT layer as

$$H^t_{mn,D,0} = ||_{\forall head_a\in\mathcal{H}} H^t_{mn,D,a,0}, \forall D\in\mathcal{N}, \tag{19}$$

where $\mathcal{H}$ is the set of all attention heads, and $||$ denotes concatenation.

In subsequent layers, message passing operates on the representations generated by the preceding layer. While each GT layer aggregates information only from the immediate neighbors, stacking multiple layers progressively expands the receptive field, enabling indirect aggregation from more distant nodes. Under the proposed hybrid decision-making architecture with two-tier access levels, the centralized agent is granted global network visibility, allowing full-graph information propagation and feature aggregation. In contrast, the distributed agent is restricted to observing its 1-hop local environment. In addition, we assume that the agent can maintain a hierarchical network view via structured observation at different scales. Consequently, the final representation of vertex l_{mn} regarding destination D as perceived by the centralized agent is

$$H^t_{mn,D} = ||_{0\leq\ell\leq\ell_{\text{total}}-1} H^t_{mn,D,\ell}, \tag{20}$$

where ℓ_{total} denotes the total number of GT layers. In contrast, from the perspective of the distributed agent for link l_{mn}, the final representation of vertex l_{mn} concerning destination D is derived only from the first GT layer, expressed as

$$\hat{H}^t_{mn,D} = H^t_{mn,D,0}. \tag{21}$$

V. Proposed Algorithm: Training and Deployment Strategy

This section first presents the counterfactual-baseline comparative training framework, then details the extended training stage for efficient deployment, and finally provides a summary of the overall algorithmic workflow.

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on January 18,2026 at 16:33:05 UTC from IEEE Xplore. Restrictions apply.

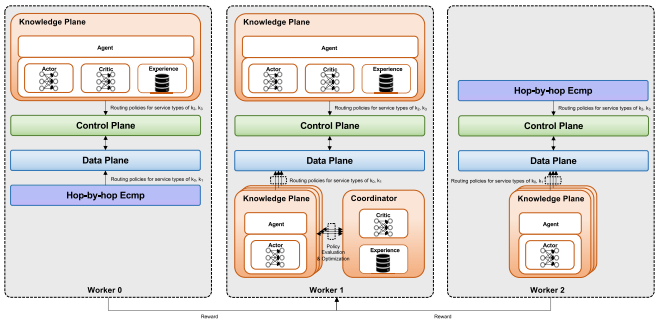


Fig. 3. Illustration of the counterfactual-baseline comparative training framework (detailed in Section IV-F), where worker 0 marginalizes the role of the distributed agents, worker 2 marginalizes the role of the centralized agent, and worker 1 follows the proposed hybrid decision-making framework.

A. Counterfactual-Baseline Comparative Training Framework

Fig. 3 illustrates the proposed counterfactual-baseline comparative training framework, based on the actor-critic architecture. The rationale behind this design originates from the hybrid decision-making architecture in this work, where the final routing policy is jointly determined by the centralized (controller-side) agent and the distributed (device-side) agents, and the reward computed via (15) inherently reflects the combined effectiveness of both agent types. During the early stages of exploration, this composite reward introduces ambiguity, as neither agent can accurately attribute reward improvements to its own policy updates. The resultant unclear policy gradients are likely to result in convergence failure or entrapment in sub-optimal solutions.

To address this, we introduce differential rewards for agent credit assignment, inspired by the counterfactual methodologies in [37]. Given that routing is inherently an end-to-end cooperative decision-making process across intermediate forwarding nodes, we assume it is feasible for distributed agents to share a globally identical reward signal.

On this basis, while Worker 1 serves as the proposed DRL-based routing model, we additionally construct two comparative workers. To be specific, Worker 0 marginalizes the role of the distributed intelligence while preserving the centralized agent's policy, and Worker 2 does the opposite by marginalizing the centralized agent while maintaining the distributed agents' behaviors. These counterfactual baselines isolate the individual contributions of each agent type. To further guide efficient policy exploration, we configure the baseline routing algorithm as a hop-by-hop Ecmp strategy, where each agent selects the next hop with the minimum hop-count difference to the destination.

Let r_0^t, r_1^t, and r_2^t denote the rewards of Worker 0, Worker 1, and Worker 2, respectively. The reward assigned to the centralized agent in Worker 1 is then expressed as

$$r_c^t = \underbrace{\gamma_0 \ln\left(1 + \frac{r_1^t}{1 \times 10^{-6} + r_2^t}\right)}_{relative} + \underbrace{\gamma_1 r_1^t}_{absoulte}, \tag{22}$$

where γ_0 and γ_1 are weighting coefficients. The logarithmic term provides a numerically stable estimate of the relative performance gain over the counterfactual baseline (Worker 2), guiding the direction of policy evolution. Meanwhile, the original reward r_1^t ensures direct optimization of the primary objective function. Accordingly, the global-sharing reward for the distributed agents in Worker 1 is computed as follows. Taking the agent associated with link l_{mn} as an example, the reward at time step t is given by

$$r_{mn}^t = \underbrace{\gamma_0 \ln\left(1 + \frac{r_1^t}{1 \times 10^{-6} + r_0^t}\right)}_{relative} + \underbrace{\gamma_1 r_1^t}_{absoulte}. \tag{23}$$

In addition, considering the cooperative nature of the distributed agents, we adopt a multi-actor shared-critic (MASC) architecture for training the distributed agents in the data plane. By collecting experience from multiple actors, the shared critic is provided with a more unbiased and comprehensive perspective for value estimation. Since all distributed agents pursue a common objective, it can further facilitate a more stable and efficient convergence. In general, the MASC architecture enhances learning efficiency while reducing memory consumption by eliminating the need for multiple critics, without additional communication overhead compared to the conventional one-to-one actor–critic architecture.

B. Extended Training for Efficient Deployment

In this section, we introduce a two-stage extended training process designed to determine when the DRL-based routing algorithm should be activated and which links' weights should be adjusted from traditional binary weights (as used in hop-by-hop Ecmp) to probabilistic weights. The fundamental objective is to reduce the overhead associated with policy inference and flow table updates, as well as to minimize potential network disturbances caused by frequent flow rerouting.

To achieve vertical intermittent activation of DRL-based routing algorithms, we initially explore a DRL-based discriminator, where an additional lightweight DRL model is established to determine whether to engage a pre-trained hybrid decision-making DRL-based routing model, conditioned on traffic dynamics. However, this design faces significant challenges. Empirical studies reveal that the learning exhibit a pronounced imbalance across different traffic intensities. Under low-traffic conditions, the performance gap between hop-by-hop Ecmp and DRL-based routing policies is marginal, resulting in a smoother searching space where the discriminator benefits from stable gradient directions and is more likely to converge rapidly to an acceptable policy. In contrast, high-traffic scenarios exhibit substantial reward differentials, requiring considerably more training iterations for the discriminator to identify that the flexibility offered by DRL-based probabilistic routing yields superior performance. This inconsistency naturally introduces asynchronous convergence behavior. While it exerts minimal influence confronting the continuous searching space, the resulting policy bias becomes particularly pronounced in the binary decision space. As a result, the discriminator experiences a twisted learning dynamics, termed policy polarization, overfitting to a specific traffic pattern and failing to adapt across varying traffic intensities.

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on January 18,2026 at 16:33:05 UTC from IEEE Xplore. Restrictions apply.

To overcome these limitations, we adopt a supervised learning paradigm to establish the mapping between traffic intensities and optimal routing strategies. Specifically, we first introduce a binary labeling process that reflects the empirical performance superiority of either hop-by-hop Ecmp or the DRL-based routing algorithm across a range of traffic intensities. A compact discriminator is then optimized using cross-entropy loss, offering improved training stability and lower inference complexity. For deployment efficiency, the discriminator is placed within the knowledge plane associated with the control plane, leveraging global link utilization information, i.e., $\{u_{mn}^t\}, \forall l_{mn} \in \mathcal{L}$.

Furthermore, we introduce a DRL-based horizontal selector to identify critical links for targeted weight updates. Inspired by SAGPool, which performs structure-aware pooling by ranking vertex importance through graph convolution, we adopt a graph convolution network (GCN) as the actor network to prioritize link-centric vertices. Importantly, the selector is activated only when the vertical discriminator determines that DRL-based routing is necessary, thereby avoiding interference under hop-by-hop Ecmp-dominated traffic conditions. As discussed in Section IV-C, the probabilistic routing paradigm requires weight normalization across all outgoing links before next-hop selection. Therefore, although the selector operates on links, final weight substitutions must respect node-level grouping of adjacent links to maintain normalization consistency. In this sense, the identification of critical links is a global optimization task, and thus, the horizontal selector is also executed centrally at the controller. The key components of the selector are defined as follows:

$$\begin{aligned} o_s^t &= o_c^t = \left\{H_{mn,D}^t\right\}, \forall l_{mn} \in \mathcal{L}, \forall D \in \mathcal{N}, \\ a_s^t &= \left\{\mathcal{I}_{mn,D}^t\right\}, \forall l_{mn} \in \mathcal{L}, \forall D \in \mathcal{N}, \\ r_s^t &= -\max_{0 \le i \le 3} \left\{1 - \eta_i^t\right\}, \end{aligned} \tag{24}$$

where $\mathcal{I}_{mn,D}^t$ denotes the importance of link l_{mn}.

C. Algorithm Workflow

In this work, the overall decision-making system comprises a centralized and L distributed knowledge planes. Within the centralized knowledge plane, we define the discriminator, graph transformer network, selector, and routing agent as $\mathcal{D}(\theta_D)$, $\mathcal{G}_c(\theta_{G_c})$, $\mathcal{S}(\cdot)$, and $\mathcal{A}_c$, respectively. The selector includes an actor network $\phi_s(\theta_{\phi_s})$ and a critic network $\psi_s(\theta_{\psi_s})$. The centralized agent $\mathcal{A}_c$ consists of an actor network $\phi_c(\theta_{\phi_c})$, a critic network $\psi_c(\theta_{\psi_c})$ and an experience buffer $\mathcal{B}_c$. Similarly, each distributed knowledge plane associated with a specific link l_{mn} contains a local graph transformer network $\mathcal{G}_{mn}(\theta_{G_{mn}})$ and an agent $\mathcal{A}_{mn}$, which includes an actor network $\phi_{mn}(\theta_{\phi_{mn}})$. All distributed knowledge planes share a common global critic network $\psi_d(\theta_{\psi_d})$ and an experience buffer $\mathcal{B}_d$. The buffers used in the extended training process are collectively referred to as $\mathcal{B}$.

The training process employs a counterfactual-baseline comparative training paradigm (illustrated in Fig. 3), involving three parallel workers. The entire training process is organized episodically to reduce memory consumption, segmented into E episodes, with each episode comprising T consecutive time steps. Upon the completion of each episode, the truncated trajectory of length T is utilized for network updates.

At each time step t, the operations across the three workers share structural similarities but differ in the scope of network view and the deployment of decision-making intelligence.

- Worker 0 only adopts DRL-based routing intelligence for the centralized controller. Based on the global network view, the graph transformer network $\mathcal{G}_c(\theta_{G_c})$ performs message passing over the entire topology. The resulting high-level representations $\left\{H_{mn,D}^t\right\}, \forall l_{mn} \in \mathcal{L}, \forall D \in \mathcal{N}$ are then fed into the actor network $\phi_c(\theta_{\phi_c})$ within the centralized agent $\mathcal{A}_c$, which outputs probabilistic link weights $\left\{w_{i,mn,D}^t\right\}, 2 \le i \le 3, \forall l_{mn} \in \mathcal{L}, \forall D \in \mathcal{N}$. For better generalization, the actor network conducts element-wise inference. The final routing policy is a combination of these learned weights and binary weights following the hop-by-hop Ecmp strategy.
- Worker 1 integrates both centralized and distributed decision-making intelligence. It inherits the global procedure from Worker 0 to generate weights for service type k_2 and k_3. Simultaneously, for each link l_{mn}, a dedicated local graph transformer $\mathcal{G}_{mn}(\theta_{G_{mn}})$ extracts $\left\{\hat{H}_{mn,D}^t\right\}, \forall D \in \mathcal{N}$ from 1-hop neighborhood observations. These features are processed by the corresponding actor network $\phi_{mn}(\theta_{\phi_{mn}})$ within the decentralized agent $\mathcal{A}_{mn}$, generating link weights $\left\{w_{i,mn,D}^t\right\}, 0 \le i \le 1, \forall D \in \mathcal{N}$. As a result, the complete routing decision fuses both global and local intelligence.
- Worker 2 operates intelligence only on distributed devices. Each $\mathcal{G}_{mn}(\theta_{G_{mn}})$ and $\phi_{mn}(\theta_{\phi_{mn}})$ generates $\left\{w_{i,mn,D}^t\right\}, 0 \le i \le 1, \forall D \in \mathcal{N}$ in parallel, as in Worker 1. In contrast, the weights for k_2 and k_3 are not learned yet determined using the baseline hop-by-hop Ecmp strategy.

Afterward, in all workers, the inferred link weights undergo an exponential mapping followed by group-wise normalization to determine the final next-hop forwarding decision and whether to trigger packet sacrifice. Subsequently, the resulting routing policies interact with the shared network environment. Responsive to environmental variations, agent(s) in each worker receive a reward r^t and updated network observations. Specifically, we derive the reward signals for the centralized and distributed agents using immediate values r_0^t, r_1^t, and r_2^t via (22) and (23). Among the three workers, only Worker 1 contributes experience tuples to the replay buffer for future retrieval. In detail, the tuple of $\left\langle o_c^t, a_c^t, r_c^t, o_c^{t+1}\right\rangle$ is stored in the buffer $\mathcal{B}_c$, while tuples of $\left\langle o_{mn}^t, a_{mn}^t, r_{mn}^t, o_{mn}^{t+1}\right\rangle, \forall l_{mn} \in \mathcal{L}$ are stored in the buffer $\mathcal{B}_d$.

At the end of each episode, all neural networks are updated by stochastic sampling from the accumulated experience tuples. To alleviate the burden of hyperparameter tuning, the training process adopts the state-of-the-art DRL paradigm, proximal policy optimization (PPO) [38], which is known for its robustness to hyperparameter selection. Within the PPO framework, the critic network estimates the value function associated with the policy generated by the actor network.

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on January 18,2026 at 16:33:05 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1 Formal Training Process of the Proposed DRL-Based Routing Algorithm

Worker 0 and Worker 1: Init $\mathcal{G}_c$, ϕ_c, ψ_c, $\mathcal{B}_c$ for centralized knowledge plane;
Worker 1 and Worker 2: Init $\mathcal{G}_{mn}$, ϕ_{mn} for each link l_{mn}, as distributed knowledge planes;
Worker 1: Init shared critic ψ_d and buffer $\mathcal{B}_d$ for distributed knowledge planes;
/**All workers operate in parallel**/
for $episode = 1$ to E **do**
  for $t = 1$ to T **do**
    In Worker 0 and Worker 1, the graph transformer network $\mathcal{G}_c(\theta_{G_c})$ abstracts high-level representations $\left\{H^t_{mn,D}\right\}, \forall l_{mn} \in \mathcal{L}, \forall D \in \mathcal{N}$ from global raw features uploaded by the management plane;
    In Worker 1 and Worker 2, for each link l_{mn}, a dedicated local graph transformer $\mathcal{G}_{mn}(\theta_{G_{mn}})$ extracts $\left\{\hat{H}^t_{mn,D}\right\}, \forall D \in \mathcal{N}$ from 1-hop neighborhood observations obtained from the management plane;
    In Worker 0 and Worker 1, the centralized actor network $\phi_c(\theta_{\phi_c})$ conducts element-wise inference, mapping $\left\{H^t_{mn,D}\right\} \rightarrow \left\{w^t_{i,mn,D}\right\}, 2 \leq i \leq 3, \forall l_{mn} \in \mathcal{L}, \forall D \in \mathcal{N}$;
    In Worker 1 and Worker 2, for each link $l_{mn} \in \mathcal{L}$, the distributed actor network $\phi_{mn}(\theta_{\phi_{mn}})$ generates link weights $\left\{w^t_{i,mn,D}\right\}, 0 \leq i \leq 1, \forall D \in \mathcal{N}$ based on corresponding observation $\left\{\hat{H}^t_{mn,D}\right\}, \forall D \in \mathcal{N}$;
    In Worker 0, issue learned probabilistic weights for service type k_2 and k_3 and binary Ecmp weights for k_0 and k_1 to the data plane;
    In Worker 1, centralized and distributed knowledge planes issue routing polices to the data plane;
    In Worker 2, issue learned probabilistic weights for service type k_0 and k_1 and binary Ecmp weights for k_2 and k_3 to the data plane;
    In all data planes, learned link weights are exponentiated and group-normalized to determine next-hop decisions and packet sacrifice triggers;
    In all workers, the final forwarding decisions interact with the environment, yielding r^t_0, r^t_1, r^t_2, and updated network observations;
    In Worker 1, calculate r^t_c and r^t_d via (22) and (23). The tuple of $\left\langle o^t_c, a^t_c, r^t_c, o^{t+1}_c \right\rangle$ is stored in the buffer $\mathcal{B}_c$, and the tuples of $\left\langle o^t_{mn}, a^t_{mn}, r^t_{mn}, o^{t+1}_{mn} \right\rangle, \forall l_{mn} \in \mathcal{L}$ are stored in the buffer $\mathcal{B}_d$;
  end for
  Update all network parameters in Worker 1 using the gradient of objectives in (27);
  Distribute updated network parameters to Worker 0 and Worker 2;
  Clear $\mathcal{B}_c \leftarrow \emptyset$, $\mathcal{B}_d \leftarrow \emptyset$;
end for

The objectives of the critic network $\psi_c(\theta_{\psi_c})$ and $\psi_d(\theta_{\psi_d})$ are respectively given by

$$J^t_{\psi_c} = \frac{1}{T}\sum_{t\in\mathcal{T}} \left(r^t_c + \gamma V(o^{t+1}_c) - V(o^t_c)\right),$$
$$J^t_{\psi_d} = \frac{1}{T}\sum_{t\in\mathcal{T}} \left(r^t_d + \gamma V(o^{t+1}_{mn}) - V(o^t_{mn})\right), \tag{25}$$

where $\mathcal{T}$ represents the set of time step indices, $\gamma \in (0,1)$ serves as a discounted factor to balance the immediate and future rewards, and $V(\cdot)$ denotes the estimated value of the observations.

In contrast to the critic, the actor network is designed to optimize long-term cumulative rewards. To enhance training stability, the PPO paradigm adopts a clipped surrogate objective, formulated as

$$J^t_{\phi_c} = \frac{1}{T}\sum_{t\in\mathcal{T}} \min\left[\rho_c A^t_c, clip\left(\rho_c, 1-\epsilon_c, 1+\epsilon_c\right) A^t_c\right],$$
$$J^t_{\phi_{mn}} = \frac{1}{T}\sum_{t\in\mathcal{T}} \min\left[\rho_{mn} A^t_{mn}, clip\left(\rho_{mn}, 1-\epsilon_{mn}, 1+\epsilon_{mn}\right) A^t_{mn}\right], \tag{26}$$

where $\rho_c = \frac{\pi_c(a|o)}{\pi^{\text{old}}_c(a|o)}$ and $\rho_{mn} = \frac{\pi_{mn}(a|o)}{\pi^{\text{old}}_{mn}(a|o)}$ quantify the policy shifts between successive updates. $A^t_c = Q(o^t_c, a^t_c) - V(o^t_c)$ and $A^t_{mn} = Q(o^t_{mn}, a^t_{mn}) - V(o^t_{mn})$ represent the advantage functions, quantifying the relative benefit of a specific action over the baseline. The clipping thresholds ϵ_c and ϵ_{mn} restrict the extent of policy updates within a bounded range. Overall, the formulations of $J^t_{\phi_c}$ and $J^t_{\phi_{mn}}$ promote conservative policy exploration, thereby mitigating the risks of falling into local optimum or divergence.

Since each knowledge plane is optimized in an end-to-end manner, the overall training objectives are defined as

$$J^t_c = \sigma^0_c J^t_{\phi_c} + \sigma^1_c J^t_{\psi_c},$$
$$J^t_{mn} = \sigma^0_{mn} J^t_{\phi_{mn}} + \sigma^1_{mn} J^t_{\psi_{mn}}, \tag{27}$$

where σ^0_c, σ^1_c, σ^0_{mn}, and σ^1_{mn} are scalar coefficients balancing the contributions of actor and critic losses. The gradients of J^t_c and J^t_{mn} are then utilized to update the corresponding network parameters. Subsequently, the updated network parameters from Worker 1 are propagated to Worker 0 and Worker 2 to enable consistent policy inference in the next episode. Upon completion of the updates, all associated experience tuples are discarded, with only the latest policy parameters retained as the operative models.

After the learning processes in Worker 1 reach convergence, an extended training phase is initiated to enable efficient deployment in practical scenarios. In this stage, multiple parallel workers are instantiated across a spectrum of traffic intensities. Each traffic level is assigned to two parallel workers: one operating under a baseline hop-by-hop Ecmp strategy, and the other governed by the fully trained DRL-based routing policy. Each worker independently executes and collects indicators, including rewards and global link utilization set $\{u^t_{mn}\}$, $\forall l_{mn} \in \mathcal{L}$. At the end of each episode, the rewards of the two workers with identical traffic intensity are compared to generate binary ground-truth labels, indicating which policy yields superior performance. Meanwhile, the trainable discriminator outputs classification logits based on randomly selected global link utilization records from the two workers. The parameters of $\mathcal{D}(\theta_D)$ are then updated using a cross-entropy loss function. Similar to the formal training process, the workers possess an empty experience buffer prior to the beginning of each episode.

The training of the selector module $\mathcal{S}(\cdot)$ is performed using a single worker and is only activated at time steps where the trained discriminator considers the DRL-based routing policy to be potentially valuable. At each selected time step t, the actor network $\phi_s(\theta_{\phi_s})$ within the selector performs an element-wise interference based on high-level features extracted by the $\mathcal{G}_c(\theta_{G_c})$, generating an action a^t_s

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on January 18,2026 at 16:33:05 UTC from IEEE Xplore. Restrictions apply.

Algorithm 2 Extended Training Process for the Discriminator

For each flow rate λ_n: Init Worker $2n$ with trained DRL-based centralized and distributed routing models and a buffer $\mathcal{B}_{2n} \in \mathcal{B}$, as well as Worker $2n+1$ with hop-by-hop routing policies and a buffer $\mathcal{B}_{2n+1} \in \mathcal{B}$;
Init $\mathcal{D}(\theta_D)$ across all workers;
/**All workers operate in parallel**/
for $episode = 1$ to E_d **do**
  for $t = 1$ to T **do**
    Each worker interacts with the environment via its embedded routing model, obtains r^t and updated observations;
    Store tuple of $\left\langle \{u^t_{mn}\}, r^t \right\rangle$, $\forall l_{mn} \in \mathcal{L}$ in the buffer of each worker;
  end for
  Assign binary label for each flow rate λ_n:
  $$y_n = \begin{cases} 1, & \text{if } r^t_{2n+1} > r^t_{2n}, \\ 0, & \text{otherwise} \end{cases}$$
  For each time step t, sample aligned $\{u^t_{mn}\}$ from both $\mathcal{B}_{2n}$ and $\mathcal{B}_{2n+1}$;
  Compute classification logits $\hat{y}_n$ via discriminator $\mathcal{D}(\theta_D)$;
  Update θ_D using cross-entropy loss;
end for

Algorithm 3 Extended Training Process for the Selector

Init ϕ_s, ψ_s, and a buffer $\mathcal{B}_s \in \mathcal{B}$ for $\mathcal{S}(\cdot)$;
Load trained DRL-based centralized and distributed routing models, trained discriminator, and hop-by-hop routing policies;
for $episode = 1$ to E_s **do**
  for $t = 1$ to T **do**
    The trained discriminator $\mathcal{D}(\theta_D)$ estimates whether the DRL-based routing policy demonstrates superior performance;
    if $\mathcal{D}(\theta_D)$ indicates DRL policy is superior **then**
      Activate $\mathcal{G}_c(\theta_{G_c})$ to produce hidden features as o^t_s;
      ϕ_s maps $o^t_s \to a^t_s$, in an element-wise manner;
      Perform group-wise summation and select top-K critical nodes based on aggregated scores;
      Pass the indices of links associated with K nodes to the actor networks in each knowledge plane;
      All knowledge planes generate transformed link weights, and replace original binary link weights with the learned ones;
      Interacts with the environment with the modified link weights, receiving r^t_s and o^{t+1}_s;
      Store tuple of $\left\langle o^t_s, a^t_s, r^t_s, o^{t+1}_s \right\rangle$ in $\mathcal{B}_s$;
    else
      Interacts with the environment with the binary link weights;
    end if
  end for
  Update all network parameters referring to the updates in the formal training process;
  Clear $\mathcal{B}_s \leftarrow \emptyset$;
end for

Algorithm 4 Execution of the Proposed DRL-Based Routing Algorithm

Load trained DRL-based centralized and distributed routing models, trained discriminator, trained selector and hop-by-hop routing policies;
for $t = 1 \to \infty$ **do**
  if $\mathcal{D}(\theta_D)$ estimates DRL policy is valuable **then**
    $\mathcal{G}_c(\theta_{G_c})$ abstracts high-level representations from raw network status;
    ϕ_s infers a^t_s based on o^t_s, evaluating the relative importance of each link $l_{mn} \in \mathcal{L}$;
    Compute group-wise aggregated scores and select the top-K critical nodes;
    Activate weight inference for the links associated with the K critical nodes via ϕ_c, $\mathcal{G}_{mn}, \forall l_{mn} \in \mathcal{L}$, and $\phi_{mn}, \forall l_{mn} \in \mathcal{L}$;
    Replace weights by these learned weights;
    Interacts with the environment using the combined weight set;
  else
    Interacts with the environment using the binary weights;
  end if
end for

from the observed network status o^t_s. This action encodes the estimated importance of each link l_{mn} in the current network condition. To maintain consistency with the downstream normalization operations, the importance scores in a^t_s are grouped by their originating nodes. The K most critical nodes (with the highest aggregated importance) are then selected, and the indices of the links associated with these critical nodes are communicated to the centralized actor $\phi_c(a|o;\theta_{\phi_c})$ as well as the underlying data plane. Subsequently, both the centralized knowledge plane and the activated distributed knowledge planes conduct probabilistic weight inference for the corresponding links, which replace the original binary weight vectors. After the final forwarding action is determined and executed, the selector receives a reward signal r^t_s, and the transition tuple $\langle o^t_s, a^t_s, r^t_s, o^{t+1}_s \rangle$ is stored in a replay buffer. Since the selector $\mathcal{S}(\cdot)$ is also implemented following the PPO paradigm, both the actor network $\phi_s(\theta_{\phi_s})$ and the critic network $\psi_s(\theta_{\psi_s})$ are updated using the same methodology mentioned above.

The execution phase begins following the completion of the E-episode training, during which all agents operate exclusively based on the learned policy networks, and no further parameter updates are performed. In this phase, the management plane continuously uploads real-time observations to both the centralized and distributed knowledge planes. Upon receiving the input, the trained discriminator $\mathcal{D}(\theta_D)$ produces a binary decision indicating whether the hop-by-hop Ecmp strategy is sufficient or if a DRL-based policy should be invoked. If Ecmp is deemed adequate, a binary routing weight matrix covering all service types and destination nodes is generated and dispatched to the control plane, which subsequently configures the data plane accordingly. Otherwise, if DRL-based routing is required, the received observation is first processed by the centralized graph transformer $\mathcal{G}_c(\theta_{G_c})$ to extract abstracted topological features.

These features are then passed to both the selector $\phi_s(\theta_{\phi_s})$ and the centralized actor network $\phi_c(\theta_{\phi_c})$. The selector evaluates the relative importance of each link and identifies the top-K critical nodes based on grouped importance scores. Only the links associated with these nodes are selected for further inference by the centralized actor network; otherwise, use the baseline weight matrices. Simultaneously, the indices of the selected critical links are issued to the data plane to activate the corresponding knowledge planes. For each activated plane, the local graph transformer $\mathcal{G}_{mn}(\theta_{G_{mn}})$ and actor network $\phi_{mn}(\theta_{\phi_{mn}})$ collaboratively generate link-specific weights. The remaining inactive planes apply default binary weights for service types k_0, k_1. In this case, the ultimate forwarding actions are performed based on the combination of critical probabilistic link weights and non-critical binary weights.

It is worth noting that both the discriminator and the selector employ lightweight network architectures, introducing minimal inference overhead and thus having negligible impact on the overall decision latency of the knowledge plane. Meanwhile, they substantially reduce deployment cost and mitigate

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on January 18,2026 at 16:33:05 UTC from IEEE Xplore. Restrictions apply.

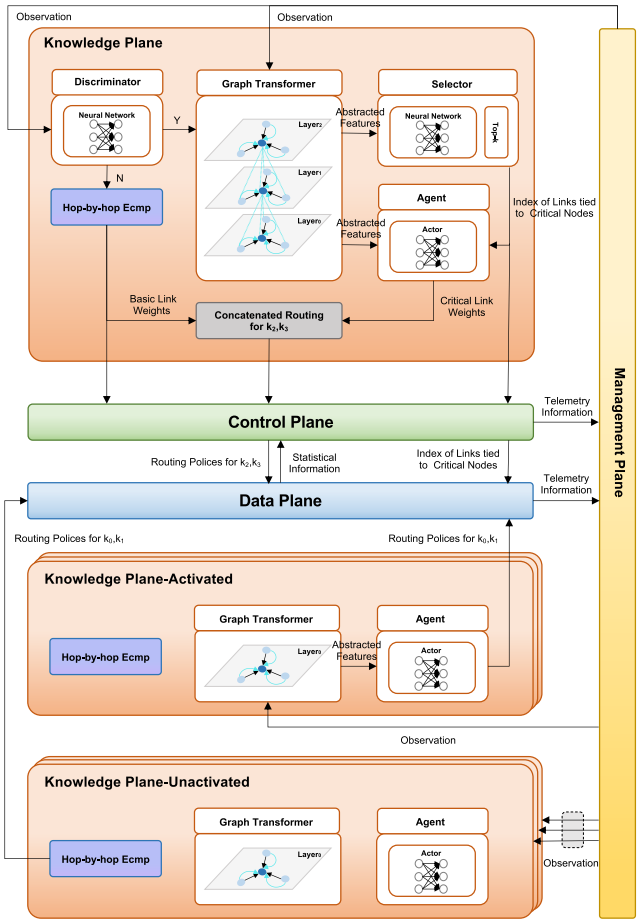


Fig. 4. Illustration of the complete decision model of the proposed algorithm during the execution phase.

unnecessary network interference. The pseudocode of the proposed algorithm is provided in **Algorithm 1**, **Algorithm 2**, **Algorithm 3**, and **Algorithm 4**. Fig. 4 illustrates the decision-making model.

D. Complexity Analysis

Based on the detailed algorithmic explanation above, the primary computational cost arises from the graph transformer network and the agent's decision-making network. Note that N denotes the number of nodes in the topology and L denotes the number of links. From the graph perspective, we further indicate the number of vertices in the abstracted graph as V and the number of edges as E. For clarity, constant factors are omitted in the time complexity analysis. Since link-level mapping is adopted, we have $V = L$ and $E < L$. Accordingly, for a centralized controller, the time complexity of the graph transformer component is $O(N(V + E))$, and that of the actor network is $O(NL)$, resulting in an overall complexity of $O(NL)$. In the case of multiple agents deployed on distributed devices, where all agents operate in parallel, the time complexity of each graph transformer component becomes $O(N(V' + E'))$. V' and E' denote the number of vertices and edges in the local subgraph for a given agent, respectively. The actor network retains a complexity of $O(N)$. Hence, the overall complexity per agent is $O(N(V' + E'))$. In summary, the total time complexity can be expressed as $O(N(V'+E'+L))$, which, given $V'+E' \leq L$, approximates to $O(NL)$. For reference, the time complexity of ECMP (introduced in Section VI-A(5)) is $O(NL)$. Therefore, although the DRL-based algorithm involves a more sophisticated design than ECMP, both approaches exhibit the same asymptotic growth in computational cost as network scale increases. This indicates that the proposed algorithm can deliver additional performance improvements in larger networks without imposing a significant computational overhead.

VI. Performance Evaluation

A. Evaluation Setup

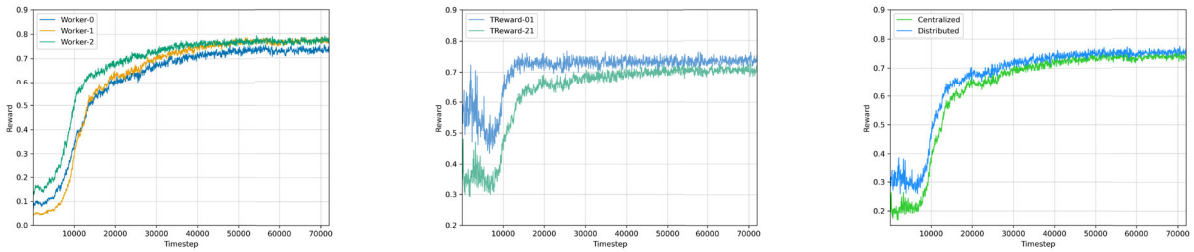
1) Implementation: In this work, the network environment is constructed using OMNeT++ 5.6.3 [39] combined with the INET library 4.2.1 [40]. The DRL agents are implemented using PyTorch [41] and Torch-Geometric [42]. Efficient communication between the agents and the network environment is achieved via ZeroMQ [43]. All simulations are executed on a server equipped with two 40-core 2.10 GHz CPUs and six Tesla T4 GPUs. In this case, the proposed algorithm requires 23h 2min for training on the default Nsfnet topology, while the average per-step inference latency during the testing phase is 2.47 ms. We further measured the inference latency on different test topologies. The average results are as follows: Abilene 2.13 ms, Harnet 2.76 ms, GridNet 2.24 ms, and Geant 2.98 ms.

2) Network Configuration: We adopt the real-world Nsfnet topology as the primary training network model. Nsfnet consists of 14 nodes and 21 bidirectional links. Due to platform-imposed constraints on data rates, the link capacities are scaled down from real-world values, with each link configured to operate at 1 Mbps and a propagation delay of 2 ms. To simulate heterogeneous traffic patterns, four service types are defined as follows:

- k_0: Packet size $p_0 = 1000$ B; delay constraint $d_0 = 2$ s. This type represents persistent traffic with continuous transmission durations uniformly sampled from $[10\ s, 20\ s]$;
- k_1: Packet size $p_1 = 150$ B; delay constraint $d_1 = 100$ ms. This type also exhibits persistent behavior with durations uniformly sampled from $[10\ s, 20\ s]$;
- k_2: Packet size $p_2 = 800$ B; delay constraint $d_2 = 2$ s. This type features intermittent traffic, alternating between active and idle phases, each uniformly distributed within $[2s, 4s]$;
- k_3: Packet size $p_3 = 50$ B; delay constraint $d_3 = 20$ ms. This type also displays intermittent burstiness with the same active/idle distribution as k_2.

To simulate realistic traffic dynamics, source nodes randomly select both the destination and service type for each new flow. The destination node and service type are chosen independently with uniform probability. Based on the above configuration, the average packet size is computed as $p_{avg} = 500$ B. The average packet generation interval is determined by $I_{avg} = \frac{p_{avg}}{\lambda}$, where λ denotes the flow rate. For each

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on January 18,2026 at 16:33:05 UTC from IEEE Xplore. Restrictions apply.



(a) Worker (b) Transformed (c) Agent

Fig. 5. Illustrations of training rewards (x-axis: timestep, y-axis: reward value).

source node, the actual interval follows a uniform distribution within $[0.9 \cdot I_{avg}, 1.1 \cdot I_{avg}]$, introducing stochastic variability. In addition, the statistical interval is set as $\tau = 2s$.[2] Similarly, packets exceeding a transmission delay of 2 s or a time-to-live (TTL) threshold of 200 hops are discarded.

3) Parallel Training: To accelerate training, we instantiate multiple parallel simulation threads, each corresponding to a distinct traffic load with flow rates $\lambda = 0.06$, 0.12, 0.18, 0.24, and 0.3 Mbps, respectively. Each simulation thread operates as an independent instance and maintains its own set of internal training workers. During training, experiences collected by all threads are aggregated into the thread with the highest traffic intensity. This centralized thread then updates all network parameters based on the aggregated experiences and subsequently disseminates the updated parameters to other threads.

4) Agent Settings: The centralized graph transformer network consists of five layers of TransformerConv modules from the Torch-Geometric library. Each layer is configured with an input dimension of 32, an output dimension of 8, and 4 attention heads. In contrast, each distributed graph transformer network comprises a single TransformerConv layer, with the same configuration as the centralized counterpart. All other networks are implemented as two-layer multi-layer perceptron (MLP). The coefficients in the reward calculation, i.e., γ_0 and γ_1, are set to 0.5. The discount factor is set to $\gamma = 0.99$. The clipping hyperparameter for the policy update is fixed at $\epsilon = 0.2$. For the weighted objective function, the coefficients are set as follows: $\sigma_c^0 = 0.5$, $\sigma_c^1 = -1$, $\sigma_{mn}^0 = 0.5$, and $\sigma_{mn}^1 = -1$. All networks are trained using the Adam optimizer [44], with a learning rate of 1×10^{-4}. The formal training phase consists of $E = 720$ episodes, each with $T = 100$ time steps. Additional extended training includes $E_d = 240$ episodes for the discriminator and $E_s = 240$ episodes for the selector module.

5) Benchmark Methods: All methods involved in the evaluation are as follows:

- **Hybrid**: The proposed hybrid decision-making DRL-based routing algorithm.
- **Vanilla**: A baseline method employs a conventional centralized knowledge plane to compute weights across all service types.
- **Ecmp**: A standard routing method that assigns binary weights (1/0) to neighbor nodes based on their proximity to the destination node.
- **Hybrid_C_Off**: A variant of the hybrid algorithm where centralized knowledge activation depends on the discriminator module's assessment.
- **Hybrid_CD_Off**: The proposed hybrid algorithm operating in intermittent mode, with both centralized and distributed planes activated only upon positive valuation by the discriminator.
- **Intelligent**: An advanced implementation of the hybrid algorithm incorporating: *(i)* discriminator-activated operation, and *(ii)* selective application of DRL-derived probabilistic weights exclusively on links connected to K critical nodes identified by the selector.
- **Random**: A comparative baseline method that randomly selects K nodes and applies DRL-generated probabilistic weights to their adjacent links, designed to evaluate the efficacy of intelligent node selection.
- **Degree**: A topology-aware benchmark that selects the K highest-degree nodes and applies DRL-based weight optimization to their connected links, providing a structural comparison to the intelligent selection mechanism.

B. Simulation Results and Analysis

To comprehensively evaluate the performance of the proposed algorithm, we examine several critical attributes, including convergence, advantages in satisfaction rates, generalization capabilities, and deployment efficiency. The following subsections provide a detailed analysis of these aspects.

1) Convergence Performance: Fig. 5 presents the complete evolution of rewards throughout training on the Nsfnet topology, encompassing individual worker rewards (r_0^t, r_1^t, and r_2^t in Fig. 5a), transformed relative rewards within (22) and (23) in Fig. 5b, and final agent-level rewards (r_c^t and r_d^t in Fig. 5c). Among the workers, Worker 2 demonstrates the fastest reward escalation during the initial training phase and, along with Worker 1, achieves the highest convergence reward. Both the learning speed and final convergence reward of Worker 0 remain inferior to those of Worker 1 and Worker 2. This observation highlights the enhanced learning efficiency of the distributed knowledge plane in comparison to its centralized counterpart. The reason primarily lies in two factors: the reduced architectural depth of the graph transformer layers, which facilitates more accessible policy optimization,

[2]This value is derived from the assumed packet lifetime of approximately 200 hops. Such an alignment facilitates stable credit assignment and efficient simulation scheduling.

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on January 18,2026 at 16:33:05 UTC from IEEE Xplore. Restrictions apply.

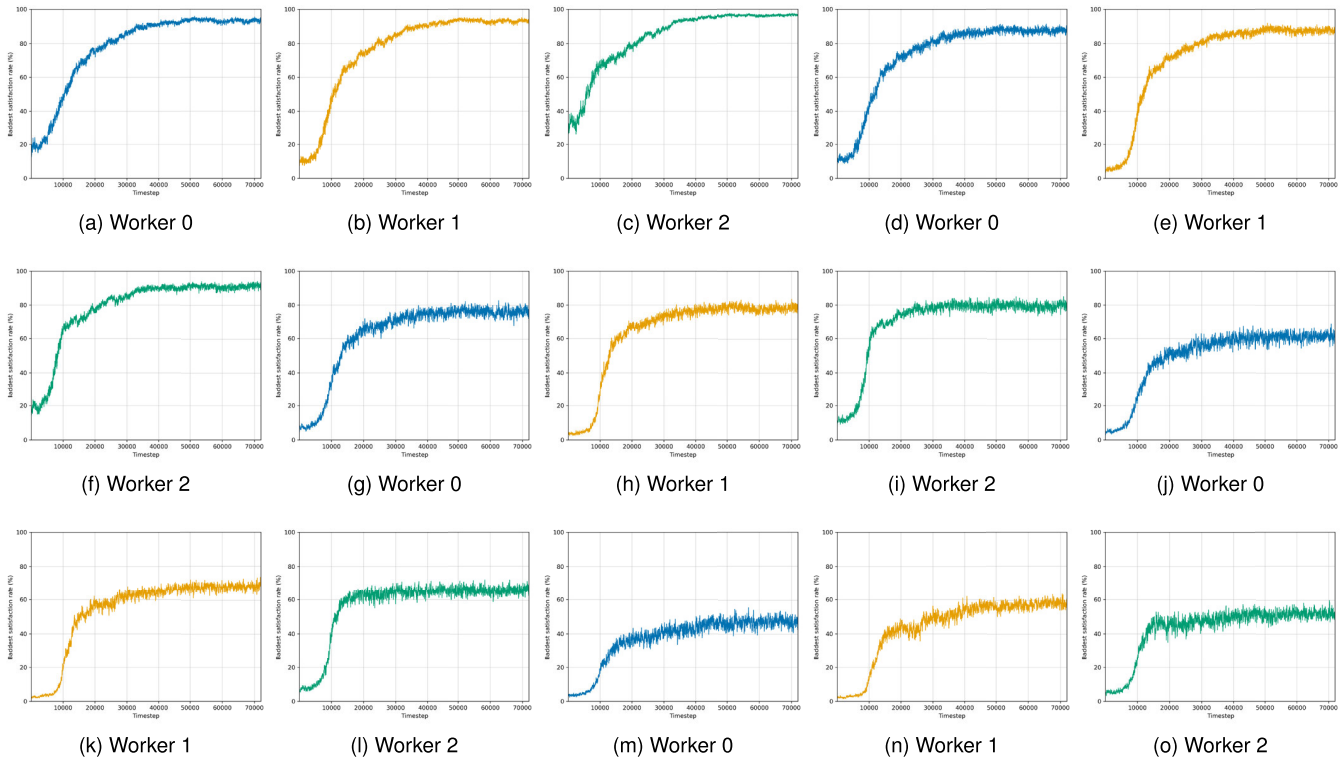


Fig. 6. Baddest satisfaction rates of the three workers across five simulation threads with varying traffic intensities (x-axis: timestep, y-axis: baddest satisfaction rate).

and the collaborative training process, which accelerates the acquisition of diverse experience tuples and promotes faster convergence. Beyond convergence speed, the superior reward attained by Worker 2 suggests that distributed intelligence offers a more substantial contribution to overall network performance than centralized intelligence. This outcome is closely tied to the algorithm's strategic reliance on selective packet sacrifice. In particular, service flows of type k_0, persistently present and exhibit low sensitivity to latency, serve as ideal candidates for such sacrifice-based decisions. As the entity responsible for managing service type k_0, the effectiveness of distributed intelligence is critical to fully realizing the potential of the proposed algorithm.

Fig. 5b illustrates the transformed relative rewards using the logarithmic mapping, including $\ln\left(1+\frac{r_1^t}{1\times10^{-6}+r_2^t}\right)$ for r_c^t (denoted as TReward-21) and $\ln\left(1+\frac{r_1^t}{1\times10^{-6}+r_0^t}\right)$ for r_d^t (denoted as TReward-01). Consistent with the observation in Fig. 5a, TReward-01 exhibits a faster ascent and better convergence. Given the properties of the logarithmic transformation employed, a transformed reward exceeding 0.7 corresponds to an original ratio greater than 1, providing quantitative evidence that converged distributed agents outperform Ecmp and the centralized intelligence demonstrates a performance roughly comparable to Ecmp. It is worth noting that Fig. 5b displays the averaged results across multiple simulation threads. A detailed evaluation of the centralized agent's effectiveness will be presented in a subsequent section. The fluctuations observed in the first 7000 training steps are hypothesized to result from architectural differences: Worker 0 and Worker 2, each representing a single-type intelligence, benefit from a more streamlined training process and thus demonstrate faster initial improvement. In contrast, Worker 1, which adopts a hybrid architecture, exhibits slower early learning due to increased complexity, as also evidenced in Fig. 5a. Nevertheless, the incorporation of absolute reward mitigates the instability of early training, as shown in Fig. 5c, effectively guiding the learning process toward high-performing strategies.

To provide a comprehensive evaluation of the effectiveness of centralized and distributed intelligence, Fig. 6 reports the baddest satisfaction rates for the three workers across five simulation threads with varying traffic intensities, specifically $\lambda = 0.06$, 0.12, 0.18, 0.24, and 0.3 Mbps from top to bottom. By conducting pairwise comparisons, one can discern the relative efficacy of centralized and distributed intelligence under different traffic conditions. At the lower traffic intensities of 0.06 and 0.12 Mbps, Worker 0 and Worker 1 yield comparable convergence values, both significantly lower than those of Worker 2. This suggests that under light traffic loads, distributed intelligence performs on par with Ecmp, whereas centralized intelligence underperforms. The reason is that Ecmp-employed binary weights coincide with the theoretically optimal routing strategy where redundancy is unnecessary. In contrast, the representation used in the distributed intelligence retains local hop differences due to its single-layer message passing, resulting in behavior closely aligned with Ecmp. On the other hand, the global aggregation performed by centralized intelligence tends to introduce less discriminative feature representations, potentially causing excessive detours that increase latency under light load. At $\lambda = 0.18$ Mbps, all three workers achieve similar performance, which may be the

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on January 18,2026 at 16:33:05 UTC from IEEE Xplore. Restrictions apply.

transition point where DRL-based routing begins to surpass static Ecmp strategies. As traffic intensity further increases to 0.24 and 0.3 Mbps, Worker 1 consistently achieves the highest minimum satisfaction rates, demonstrating that both forms of intelligence now significantly outperform Ecmp. Overall, distributed intelligence maintains parity with Ecmp under low traffic conditions and exhibits substantial gains at higher loads, whereas centralized intelligence lags under light traffic but outperforms Ecmp under heavy load. This trend is quantitatively confirmed in Fig. 5, where the final convergence value of TReward-01 (associated with distributed intelligence) exceeds 0.7, while TReward-21 (associated with centralized intelligence) stabilizes around 0.7.

Nevertheless, it is important to note that although distributed intelligence appears more effective based on the analysis above, centralized intelligence is non-negligible. First, centralized intelligence is responsible for managing service type k_3, which imposes the most stringent QoS constraints among all traffic types. Maximizing the minimum satisfaction rate for such flows is inherently more challenging. Second, the centralized knowledge plane offers intrinsic advantages in network administration, including simplified management, holistic awareness, and greater robustness to potential failures or adversarial disruptions. Therefore, while distributed intelligence offers substantial performance benefits, centralized control remains indispensable for strategic oversight and resilience.

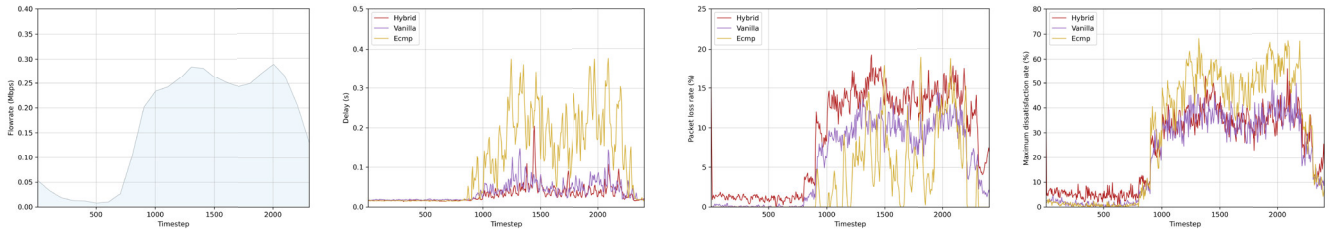
2) Advantages in Satisfaction Rates: Fig. 7 illustrates the flow rate, packet loss rate, and maximum dissatisfaction rate for three representative algorithms tested on the Nsfnet topology. The flow rate follows a 24-hour historical traffic trace collected from a subnet of our campus network, and is rescaled by a factor of 0.01 to accommodate platform bandwidth constraints. As shown in Fig. 7, the **Hybrid** achieves a comparable maximum dissatisfaction rate to **Vanilla**, while outperforming **Ecmp**. The overall behavioral pattern of **Hybrid** closely resembles that of **Vanilla**. Specifically, they experience higher packet loss compared with **Ecmp**, which may lead to a lower overall throughput. Nevertheless, they achieve better service satisfaction, which represents the primary objective of our optimization. However, **Vanilla** consistently maintains a lower packet loss rate than **Hybrid** across all traffic intensities, which contributes to superior performance in terms of maximum dissatisfaction rate, particularly under light traffic conditions. In addition, as shown in Fig. 7b, the proposed algorithm, whether implemented in the centralized (**Vanilla**) or **Hybrid** manner, consistently achieves a lower average end-to-end delay compared with **Ecmp**. Overall, although the adaptive packet sacrifice mechanism drops a small portion of traffic flows, it effectively improves the lower-bound service performance of the network without deteriorating the overall end-to-end delay.

For a more detailed comparison, Fig. 8 depict the delay, packet loss rate and dissatisfaction rate across different service types for all evaluated methods. A key observation is that **Ecmp** treats all service types uniformly, while both **Hybrid** and **Vanilla** exhibit differentiated behaviors across service types. Notably, the improvements in dissatisfaction rates for k_1 and k_3 are achieved at the cost of degraded performance in k_0 and k_2. This observation can be found in both the delay and packet loss rate, as these two metrics exhibit consistent patterns across different service types. Due to the use of a max operation in the reward formulation, the final dissatisfaction rate envelopes of k_0, k_2, and k_3 converge to similar levels, whereas k_1, which has moderate QoS requirements, consistently achieves the lowest dissatisfaction rate. Another prominent difference emerges when comparing packet loss rates. **Hybrid** allows packet drops even for k_1 and k_3, while **Vanilla** maintains zero packet loss for these flows. Interestingly, the packet loss rates of k_1 and k_3 under **Hybrid** remain relatively stable across varying traffic loads, unlike the packet loss rates of k_0 and k_2, which exhibit load-dependent variability. This suggests that the packet loss in k_1 and k_3 is not an adaptive reaction to traffic congestion but rather a structural limitation of **Hybrid**. In particular, we infer that this behavior originates from the lack of a global service-level view. Without complete knowledge of all service flows, the agent may over-allocate resources for unseen traffic, inadvertently deprioritizing the flows it is responsible for, despite their stringent QoS requirements.

It is important to highlight that although **Hybrid** exhibits the aforementioned limitation, the nearly identical dissatisfaction ratios of k_1 and k_3 under **Hybrid** and **Vanilla** indicate that its practical impact remains marginal. Moreover, within each flow category (i.e., k_0/k_1 and k_2/k_3), the agents still demonstrate QoS-awareness by prioritizing more QoS-sensitive flows and sacrificing the less sensitive ones. This reflects an inherent trade-off: while the absence of a global service-level view introduces a fixed packet loss, the hybrid architecture simultaneously provides tangible advantages by distributing computational and decision-making workloads, reducing information collection overhead, and mitigating the risks associated with single-point control. We position this study as a first step, offering insights that may guide researchers and practitioners in selecting architectures aligned with their operational objectives and deployment constraints.

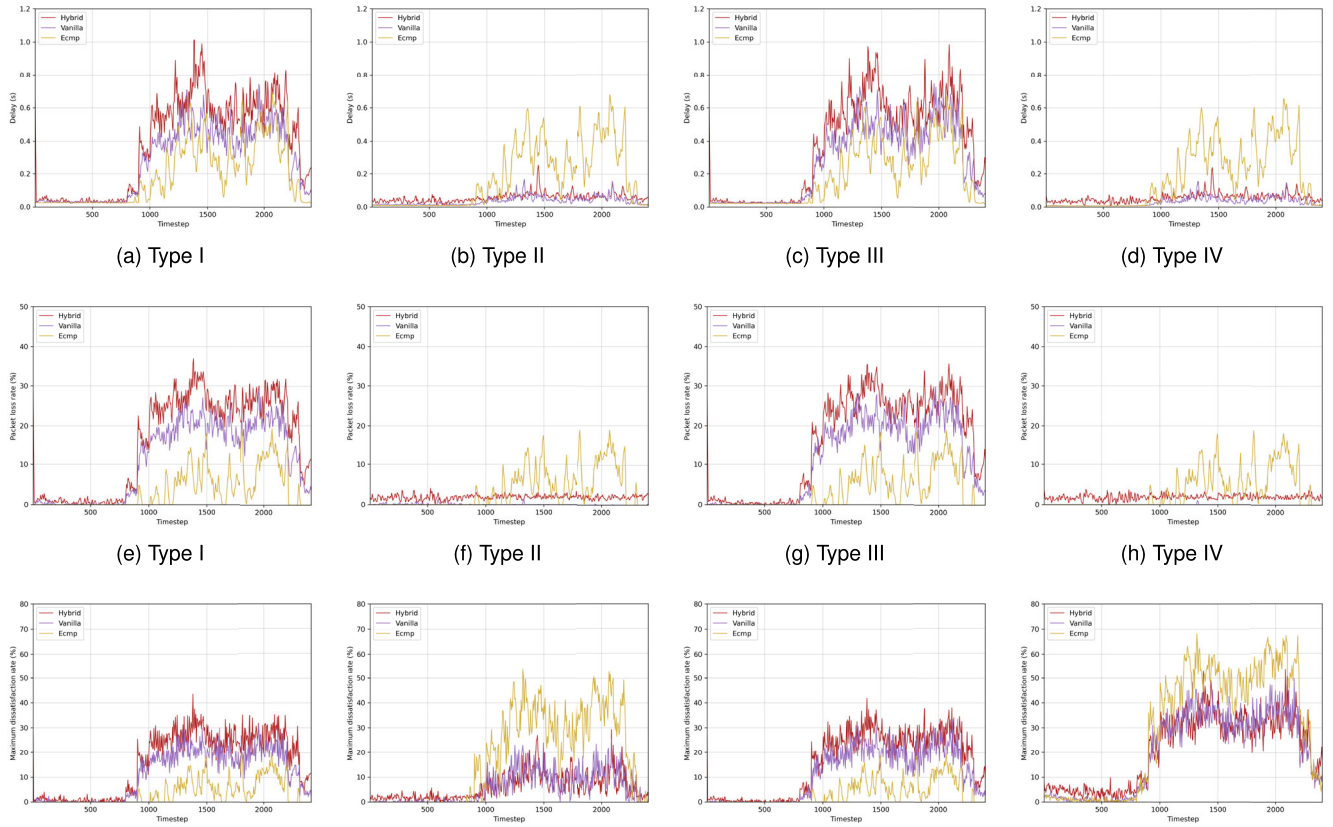
3) Generalization Across Unseen Topologies: Since full-scale training and testing in real-world networks is often infeasible due to safety and cost concerns, it is essential for models to demonstrate strong generalization capabilities on previously unseen data. To this end, we deploy the trained DRL-based routing models on three real-world topologies from the Internet Topology Zoo [45]: Abilene (11 nodes, 14 bidirectional links), Harnet (20 nodes, 26 bidirectional links), and Gridnet (9 nodes, 20 bidirectional links), representing smaller, larger, and denser network scenarios, respectively. Fig. 9 presents the maximum dissatisfaction rates across different service types for all methods on these three topologies. The simulation results show that both **Hybrid** and **Vanilla** consistently outperform **Ecmp**. Notably, **Hybrid** suffers significant performance degradation on Gridnet, while **Vanilla** maintains its performance advantage. This phenomenon can be attributed to two factors. First, Gridnet's highly interconnected topology offers multiple redundant paths, reducing the routing challenge for **Ecmp**. Second, due to the limited observation scope of distributed intelligence, neighbor nodes may appear similar

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on January 18,2026 at 16:33:05 UTC from IEEE Xplore. Restrictions apply.



(a) Flow Rate (b) Delay (c) Packet Loss Rate (d) Maximum Dissatisfaction Rate

Fig. 7. Evaluation of algorithms over the Nsfnet topology (x-axis: timestep, y-axis: flowrate/delay/packet loss rate/maximum dissatisfaction rate).

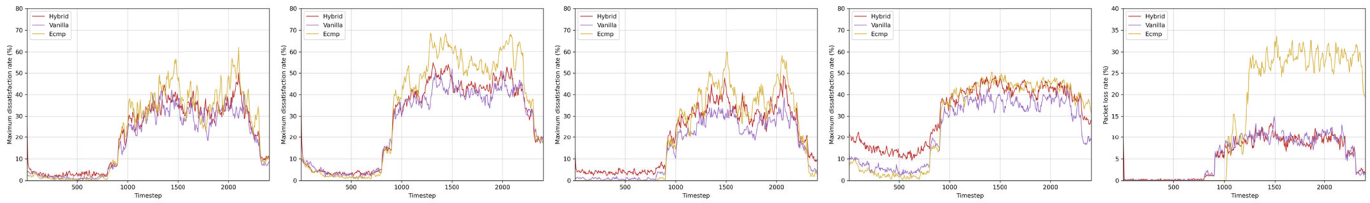


(a) Type I (b) Type II (c) Type III (d) Type IV

(e) Type I (f) Type II (g) Type III (h) Type IV

(i) Type I (j) Type II (k) Type III (l) Type IV

Fig. 8. Comparison of delay, packet loss rate, and maximum dissatisfaction rate for all methods across different service types on the Nsfnet topology (x-axis: timestep, y-axis: delay/packet loss rate/maximum dissatisfaction rate).



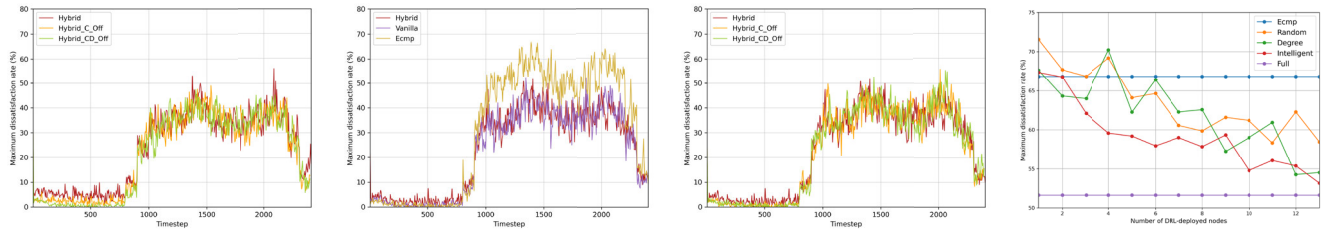
(a) Abilene (b) Harnet (c) Gridnet (d) Geant (e) Geant(loss rate)

Fig. 9. Maximum dissatisfaction rates across different types for methods tested on unseen topologies (x-axis: timestep, y-axis: maximum dissatisfaction rate).

to the agent, leading to less discriminative and suboptimal forwarding decisions.

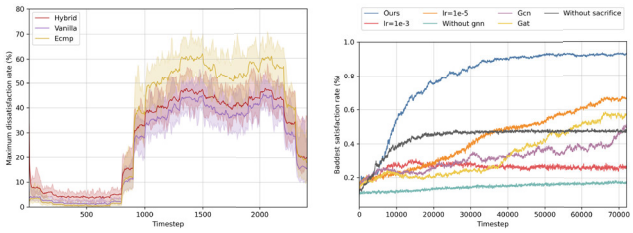
To better approximate modern ISP networks, we further evaluate the proposed algorithm on the large-scale Geant topology (40 nodes, 61 edges) from Internet topology zoo. As shown in Fig. 9, the performance gap between the DRL-based algorithms and ECMP narrows significantly: **Hybrid** even falls behind **Ecmp** under low traffic, and performs comparably at high traffic. We attribute this to the limited 1-hop observation space of distributed agents in a topology with diameter 8. Noticing the overall flattening of performance curves compared with smaller topologies,

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on January 18,2026 at 16:33:05 UTC from IEEE Xplore. Restrictions apply.



(a) (b) (c) (d)

Fig. 10. Maximum dissatisfaction rate across different service types for extended scenarios on the Nsfnet topology (x-axis: timestep, y-axis: maximum dissatisfaction rate/packet loss rate).



(a) Unseen link failures (b) Variants comparison

Fig. 11. Evaluation under link failures and variant training comparison (x-axis: timestep, y-axis: maximum dissatisfaction rate/baddest satisfaction rate).

we further analyse the loss rate. Simulation result shows that although the peak dissatisfaction rates of **Ecmp** are lower than those in Nsfnet, its packet loss already exceeds 30%, indicating a substantially different network environment. Since DRL is inherently experience-driven, its performance degradation in such conditions is understandable. These results demonstrate that while the proposed algorithm exhibits generalization capability, its effectiveness diminishes as the gap between training and deployment environments widens, eventually approaching **Ecmp**, in which case retraining may be required.

4) Deployment Efficiency: We then evaluate the effectiveness of the proposed discriminator and selector modules. Fig. 10a illustrates the maximum dissatisfaction rates of **Hybrid** and its two intermittently activated variants. According to the statistics, under the judgment of the discriminator, **Hybrid_C_Off** activates the DRL-based routing module for only 44.38% of the total run time, while **Hybrid_CD_Off** does so for 55.92% of the time. Despite significantly reduced activation durations, both variants achieve comparable performance to the fully activated **Hybrid** under high traffic loads. Moreover, due to the discriminator's ability to identify DRL-based routing is unnecessary in low-traffic scenarios, both **Hybrid_C_Off** and **Hybrid_CD_Off** yield even lower dissatisfaction rates in such conditions.

To evaluate the effectiveness of the proposed selector, we further conduct training under non-uniform traffic conditions. While the average flow rate remains the same as in previous experiments, the flow rate of each source node is scaled by a random factor. Fundamental performance results under this setting are presented in Fig. 10b and Fig. 10c. Fig. 10d demonstrates the maximum dissatisfaction rates of **Hybrid** under varying numbers of critical nodes selected for partial deployment. Note that partial deployment is only applied when the algorithm is activated. The results clearly show that the proposed selector consistently outperforms both **Random** and **Degree**, particularly when the number of selected nodes is less than half of the total nodes in the network. Specifically, deploying the model on only 4 out of 14 nodes achieves over 50% of the total performance gain, operating for merely 60% of the total runtime.

5) Robustness Evaluation: We further evaluate the robustness of the proposed algorithm to assess its practical feasibility. Specifically, the DRL-based algorithms are trained under a scenario where the link with the highest betweenness experiences a failure (halved bandwidth and doubled propagation delay) and tested under unseen failures on other links. Fig. 11a presents the simulation results, with solid lines indicating the mean and shaded areas representing the variance across different link failures. The results show that the proposed DRL-based algorithms exhibit strong robustness, with **Hybrid** and **Vanilla** perform similarly, consistently achieving lower maximum dissatisfaction rates than **Ecmp** with relatively small fluctuations.

6) Comparison Among Variants: To evaluate the effectiveness of the proposed algorithm and its components, we conduct experiments on learning rates, GNN variants, and ablation of the GNN and packet sacrifice mechanism. Fig. 11 illustrates the training curves of the algorithm and its variants. For learning rates, 1×10^{-3} leds to rapid initial improvement but resulted in degraded final performance due to overly aggressive policy updates. A learning rate of 1×10^{-5} causes slow policy updates, preventing convergence even after 720 episodes. These results confirm that 1×10^{-4} provides a balanced and effective learning rate. Comparing GNNs, both GAT and GCN failed to converge within 720 episodes. GAT slightly outperformed GCN owing to its finer attention mechanism, yet both were clearly inferior to the graph transformer used in our algorithm. Ablation studies further show that the GNN contributes more to performance gains than the packet sacrifice mechanism, although both are essential for achieving optimal results. Overall, these experiments validate that the chosen learning rate, the graph transformer, and the packet sacrifice mechanism collectively enable the superior performance of the proposed algorithm.

VII. Conclusion

This paper presented a QoS-aware and cost-efficient routing algorithm based on graph reinforcement learning. The

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on January 18,2026 at 16:33:05 UTC from IEEE Xplore. Restrictions apply.

fundamental design lies in an adaptive packet sacrifice mechanism, which selectively drops less-sensitive flows to ensure the minimum satisfaction rate across all services. In addition, the algorithm incorporates a hybrid decision-making paradigm designed to retain a global perspective and centralized control, while alleviating computational burdens and mitigating risks associated with single-point control. Moreover, vertical intermittent activation and horizontal partial deployment are introduced via supervised learning and a SAGPool-enhanced DRL model, respectively, to minimize rerouting overhead and operational costs.

Extensive simulations confirm that the proposed algorithm outperforms the benchmarks in terms of maximum dissatisfaction rate across most scenarios, including previously unseen topologies and link failures. Notably, our evaluations reveal that over 50% of the maximum achievable performance gain can be attained through a lightweight deployment, where only 28.5% of forwarding devices perform policy inference, operating for merely 60% of the total runtime.

However, it should be noted that two limitations remain. First, on highly clustered structures such as Gridnet or super large-scale network such as Geant, the advantage becomes less evident due to the local interactions of distributed agents. Second, the restricted service view may cause a minimal but unavoidable packet loss, as resources are occasionally reserved for unseen service types.

Looking ahead, we plan to extend this work toward practical deployment in real-world networks, focusing on challenges such as synchronization overhead among controllers and hardware/software upgrades of routing devices. In this context, we intend to leverage SDN-based environments, such as Ryu and Mininet, to facilitate better alignment and reproducibility.

REFERENCES

[1] G. Malkin, *RIP Version 2*, document RFC 2453, IETF, Nov. 1998.

[2] J. Moy, *OSPF Version 2*, document RFC 2328, IETF, Apr. 1998.

[3] H. Smit and T. Li, *Intermediate System to Intermediate System (IS-IS) Extensions for Traffic Engineering (TE)*, document RFC 3784, IETF, Jun. 2004.

[4] M. Chiesa, G. Kindler, and M. Schapira, "Traffic engineering with equal-cost-MultiPath: An algorithmic perspective," *IEEE/ACM Trans. Netw.*, vol. 25, no. 2, pp. 779–792, Apr. 2017.

[5] M. Pizzutti and A. E. Schaeffer-Filho, "Adaptive multipath routing based on hybrid data and control plane operation," in *Proc. IEEE INFOCOM-IEEE Conf. Comput. Commun.*, Apr. 2019, pp. 730–738.

[6] X. Wang et al., "Deep reinforcement learning: A survey," *IEEE Trans. Neural Netw. Learn. Syst.*, pp. 1–15, Sep. 2022.

[7] A. Mendiola, J. Astorga, E. Jacob, and M. Higuero, "A survey on the contributions of software-defined networking to traffic engineering," *IEEE Commun. Surveys Tuts.*, vol. 19, no. 2, pp. 918–953, 2nd Quart., 2017.

[8] G. Stampa et al., "A deep-reinforcement learning approach for software-defined networking routing optimization," 2017, *arXiv:1709.07080*.

[9] R. Amin, M. Reisslein, and N. Shah, "Hybrid SDN networks: A survey of existing approaches," *IEEE Commun. Surveys Tuts.*, vol. 20, no. 4, pp. 3259–3306, 4th Quart., 2018.

[10] A. Mestres et al., "Knowledge-defined networking," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 47, no. 3, pp. 2–10, 2017.

[11] Y. Song, J. Zhou, X. Li, and J. Liu, "Enabling robust and generalized DRL-driven routing optimization via exploration of graph neural networks," *IEEE Trans. Cognit. Commun. Netw.*, early access, Feb. 20, 2025, doi: 10.1109/TCCN.2025.3544276.

[12] S. Blake, D. Black, M. Carlson, E. Davies, Z. Wang, and W. Weiss, "Rfc2475: An architecture for differentiated service," Tech. Rep., 1998.

[13] C. Semeria, "Supporting differentiated service classes: Queue scheduling disciplines," *Juniper Netw.*, pp. 11–14, Mar. 2001.

[14] Q. Ma and P. Steenkiste, "Supporting dynamic inter-class resource sharing: A multi-class QoS routing algorithm," in *Proc. IEEE INFOCOM Conf. Comput. Commun.*, vol. 2, Jun. 1999, pp. 649–660.

[15] S. Bera, S. Misra, and A. Jamalipour, "FlowStat: Adaptive flow-rule placement for per-flow statistics in SDN," *IEEE J. Sel. Areas Commun.*, vol. 37, no. 3, pp. 530–539, Mar. 2019.

[16] N. Saha, S. Bera, and S. Misra, "Sway: Traffic-aware QoS routing in software-defined IoT," *IEEE Trans. Emerg. Topics Comput.*, vol. 9, no. 1, pp. 390–401, Jan. 2021.

[17] H. Huang, S. Guo, P. Li, B. Ye, and I. Stojmenovic, "Joint optimization of rule placement and traffic engineering for QoS provisioning in software defined network," *IEEE Trans. Comput.*, vol. 64, no. 12, pp. 3488–3499, Dec. 2015.

[18] C. Yu, J. Lan, Z. Guo, and Y. Hu, "DROM: Optimizing the routing in software-defined networks with deep reinforcement learning," *IEEE Access*, vol. 6, pp. 64533–64539, 2018.

[19] P. Sun, Y. Hu, J. Lan, L. Tian, and M. Chen, "TIDE: Time-relevant deep reinforcement learning for routing optimization," *Future Gener. Comput. Syst.*, vol. 99, pp. 401–409, Oct. 2019.

[20] P. T. A. Quang, Y. Hadjadj-Aoul, and A. Outtagarts, "Deep reinforcement learning based QoS-aware routing in knowledge-defined networking," in *Proc. Int. Conf. Heterogeneous Netw. Quality Rel. Secur. Robustness*, 2018, pp. 14–26.

[21] D. M. Casas-Velasco, O. M. C. Rendon, and N. L. S. Da Fonseca, "DRSIR: A deep reinforcement learning approach for routing in software-defined networking," *IEEE Trans. Netw. Service Manage.*, vol. 19, no. 4, pp. 4807–4820, Dec. 2022.

[22] Y. Xiao, Y. Yang, H. Yu, and J. Liu, "Scalable QoS-aware multipath routing in hybrid knowledge-defined networking with multiagent deep reinforcement learning," *IEEE Trans. Mobile Comput.*, vol. 23, no. 11, pp. 10628–10646, Nov. 2024.

[23] T. Dong et al., "Generative adversarial network-based transfer reinforcement learning for routing with prior knowledge," *IEEE Trans. Netw. Service Manage.*, vol. 18, no. 2, pp. 1673–1689, Jun. 2021.

[24] C. Liu, M. Xu, Y. Yang, and N. Geng, "DRL-OR: Deep reinforcement learning-based online routing for multi-type service requirements," in *Proc. IEEE INFOCOM-IEEE Conf. Comput. Commun.*, May 2021, pp. 1–10.

[25] M. Ye, J. Zhang, Z. Guo, and H. J. Chao, "FlexDATE: Flexible and disturbance-aware traffic engineering with reinforcement learning in software-defined networks," *IEEE/ACM Trans. Netw.*, vol. 31, no. 4, pp. 1433–1448, Aug. 2023.

[26] M. Ye, Y. Hu, J. Zhang, Z. Guo, and H. J. Chao, "Mitigating routing update overhead for traffic engineering by combining destination-based routing with reinforcement learning," *IEEE J. Sel. Areas Commun.*, vol. 40, no. 9, pp. 2662–2677, Sep. 2022.

[27] J. Zhang, M. Ye, Z. Guo, C.-Y. Yen, and H. J. Chao, "CFR-RL: Traffic engineering with reinforcement learning in SDN," *IEEE J. Sel. Areas Commun.*, vol. 38, no. 10, pp. 2249–2259, Oct. 2020.

[28] Y. Shi et al., "Masked label prediction: Unified message passing model for semi-supervised classification," 2020, *arXiv:2009.03509*.

[29] V. Ho-Huu, S. Hartjes, H. G. Visser, and R. Curran, "An improved MOEA/D algorithm for bi-objective optimization problems with complex Pareto fronts and its application to structural optimization," *Expert Syst. Appl.*, vol. 92, pp. 430–446, Feb. 2018.

[30] J. Lee, I. Lee, and J. Kang, "Self-attention graph pooling," in *Proc. Int. Conf. Mach. Learn.*, May 2019, pp. 3734–3743.

[31] X. Li et al., "Network slicing for 5G: Challenges and opportunities," *IEEE Internet Comput.*, vol. 21, no. 5, pp. 20–27, Sep. 2017.

[32] B. Pfaff et al., "The design and implementation of open vSwitch," in *Proc. 12th USENIX Symp. Networked Syst. Design Implement.*, 2015, pp. 117–130.

[33] F. Hu, Q. Hao, and K. Bao, "A survey on software-defined network and OpenFlow: From concept to implementation," *IEEE Commun. Surveys Tuts.*, vol. 16, no. 4, pp. 2181–2206, 4th Quart., 2014.

[34] L. Richardson and S. Ruby, *RESTful Web Services*. Sebastopol, CA, USA: O'Reilly Media, 2008.

[35] R. Hofstede et al., "Flow monitoring explained: From packet capture to data analysis with NetFlow and IPFIX," *IEEE Commun. Surveys Tuts.*, vol. 16, no. 4, pp. 2037–2064, 4th Quart., 2014.

[36] L. W. Beineke and J. S. Bagga, *Line Graphs and Line Digraphs*. Cham, Switzerland: Springer, 2021.

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on January 18,2026 at 16:33:05 UTC from IEEE Xplore. Restrictions apply.

[37] J. Foerster, G. Farquhar, T. Afouras, N. Nardelli, and S. Whiteson, "Counterfactual multi-agent policy gradients," in *Proc. AAAI Conf. Artif. Intell.*, 2018, pp. 2974–2982.
[38] J. Schulman et al., "Proximal policy optimization algorithms," 2017, *arXiv:1707.06347*.
[39] A. Varga, "Discrete event simulation system," in *Proc. Eur. Simul. Multiconference (ESM'2001)*, 2001, pp. 1–7.
[40] L. Mészáros, A. Varga, and M. Kirsche, "INET framework," in *Recent Adv. Netw. Simul., OMNeT++ Environ. Ecosystem*, 2019, pp. 55–106.
[41] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Proc. NIPS*, Dec. 2019, pp. 8024–8035.
[42] M. Fey and J. E. Lenssen, "Fast graph representation learning with PyTorch geometric," 2019, *arXiv:1903.02428*.
[43] P. Hintjens, *ZeroMQ: Messaging for Many Applications*, 2013.
[44] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2014, *arXiv:1412.6980*.
[45] S. Knight, H. X. Nguyen, N. Falkner, R. Bowden, and M. Roughan, "The Internet topology zoo," *IEEE J. Sel. Areas Commun.*, vol. 29, no. 9, pp. 1765–1775, Oct. 2011.

Yuqian Song received the B.E. degree (Hons.) in communication engineering from Beijing University of Posts and Telecommunications, Beijing, China, in 2021, where she is currently pursuing the Ph.D. degree with the School of Artificial Intelligence. Her current research interests include network traffic engineering, wireless resource allocation, vehicular networks, and reinforcement learning.

Jingli Zhou received the B.E. degree in information engineering from Beijing University of Posts and Telecommunications, Beijing, China, in 2023, where he is currently pursuing the M.E. degree in artificial intelligence. His research interests include deep reinforcement learning, graph neural networks, and network routing algorithms.

Shudan Yu received the M.E. degree in management information systems from Beijing University of Posts and Telecommunications (BUPT). Her research interests include big data analysis, deep learning, scientific research resource management, and the industrial applications of large models.

Jun Liu (Member, IEEE) received the B.E. and Ph.D. degrees from the Department of Information Engineering, Beijing University of Posts and Telecommunications (BUPT), Beijing, China, in 1998 and 2003, respectively. He is currently the Director of the Research Center for Data Science, School of Artificial Intelligence, BUPT. His research interests include telecommunication big data analysis and network management.

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on January 18,2026 at 16:33:05 UTC from IEEE Xplore. Restrictions apply.